

Dueling Network Architectures for Deep Reinforcement Learning

Ziyu Wang
Tom Schaul
Matteo Hessel
Hado van Hasselt
Marc Lanctot
Nando de Freitas

Google DeepMind, London, UK

ZIYU@GOOGLE.COM
SCHAUL@GOOGLE.COM
MTTHSS@GOOGLE.COM
HADO@GOOGLE.COM
LANCTOT@GOOGLE.COM
NANDODEFREITAS@GMAIL.COM

Abstract

In recent years there have been many successes of using deep representations in reinforcement learning. Still, many of these applications use conventional architectures, such as convolutional networks, LSTMs, or auto-encoders. In this paper, we present a new neural network architecture for model-free reinforcement learning. Our dueling network represents two separate estimators: one for the state value function and one for the state-dependent action advantage function. The main benefit of this factoring is to generalize learning across actions without imposing any change to the underlying reinforcement learning algorithm. Our results show that this architecture leads to better policy evaluation in the presence of many similar-valued actions. Moreover, the dueling architecture enables our RL agent to outperform the state-of-the-art on the Atari 2600 domain.

1. Introduction

Over the past years, deep learning has contributed to dramatic advances in scalability and performance of machine learning (LeCun et al., 2015). One exciting application is the sequential decision-making setting of reinforcement learning (RL) and control. Notable examples include deep Q-learning (Mnih et al., 2015), deep visuomotor policies (Levine et al., 2015), attention with recurrent networks (Ba et al., 2015), and model predictive control with embeddings (Watter et al., 2015). Other recent successes include massively parallel frameworks (Nair et al., 2015) and expert move prediction in the game of Go (Maddison et al., 2015), which produced policies matching those of Monte Carlo tree search programs, and squarely beaten a professional player when combined with search (Silver et al., 2016).

In spite of this, most of the approaches for RL use standard neural networks, such as convolutional networks, MLPs, LSTMs and autoencoders. The focus in these recent advances has been on designing improved control and RL algorithms, or simply on incorporating existing neural network architectures into RL methods. Here, we take an *alternative but complementary approach* of focusing primarily on innovating a neural network architecture that is better suited for model-free RL. This approach has the benefit that the new network can be easily combined with existing and future algorithms for RL. That is, this paper advances a new network (Figure 1), but uses already published algorithms.

The proposed network architecture, which we name the *dueling architecture*, explicitly separates the representation of state values and (state-dependent) action advantages. The dueling architecture consists of two streams that represent the value and advantage functions, while sharing a common

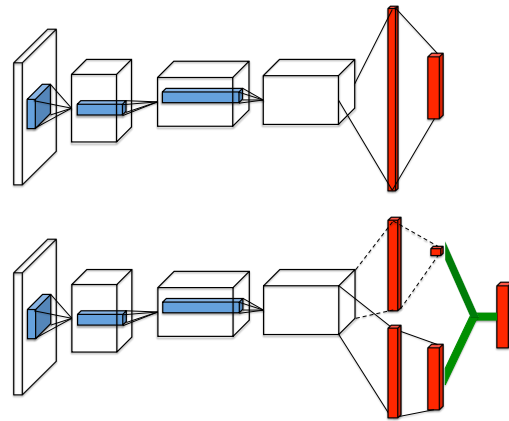


Figure 1. A popular single stream Q -network (**top**) and the dueling Q -network (**bottom**). The dueling network has two streams to separately estimate (scalar) state-value and the advantages for each action; the green output module implements equation (9) to combine them. Both networks output Q -values for each action.

深度强化学习的决斗网络架构

王子玉
汤姆·斯考尔
马泰奥·赫塞尔
哈多·范·哈瑟尔特
马克·兰克托
南多·德·弗雷塔斯
谷歌 DeepMind, 英国伦敦

子玉@GOOGLE.COM
SCHAUL@GOOGLE.COM
MTTHSS@GOOGLE.COM
HADO@GOOGLE.COM
LANCTOT@GOOGLE.COM
NANDODEFREITAS@GMAIL.COM

抽象的

近年来, 在强化学习中使用深度表示已经取得了许多成功。尽管如此, 这些应用程序中的许多都使用传统的架构, 例如卷积网络、LSTM 或自动编码器。在本文中, 我们提出了一种用于无模型强化学习的新神经网络架构。我们的决斗网络代表两个独立的估计器: 一个用于状态价值函数, 另一个用于状态相关的动作优势函数。这种分解的主要好处是泛化跨动作的学习, 而不会对底层的强化学习算法进行任何改变。我们的结果表明, 在存在许多类似价值的行动的情况下, 这种架构可以带来更好的政策评估。此外, 决斗架构使我们的 RL 代理能够超越 Atari 2600 领域的最新技术。

一、简介

在过去的几年里, 深度学习为机器学习的可扩展性和性能方面的巨大进步做出了贡献 (LeCun et al., 2015)。强化学习 (RL) 和控制的顺序决策设置是一项令人兴奋的应用。值得注意的例子包括深度 Q 学习 (Mnih 等人, 2015)、深度视觉运动策略 (Levine 等人, 2015)、循环网络的注意力 (Ba 等人, 2015) 以及嵌入的模型预测控制 (Watter 等人, 2015)。最近的其他成功包括大规模并行框架 (Nair 等人, 2015) 和围棋游戏中的专家走法预测 (Maddison 等人, 2015), 它们产生了与蒙特卡罗树搜索程序相匹配的策略, 并且在与搜索相结合时直接击败了职业棋手 (Silver 等人, 2016)。

尽管如此, 大多数强化学习方法都使用标准神经网络, 例如卷积网络、MLP、LSTM 和自动编码器。这些最新进展的重点是设计改进的控制和强化学习算法, 或者只是将现有的神经网络架构合并到强化学习方法中。在这里, 我们主要关注创新更适合无模型强化学习的神经网络架构。这种方法的好处是新网络可以轻松地与现有和未来的强化学习算法相结合。也就是说, 本文提出了一种新的网络 (图 1), 但使用了已经发布的算法。

所提出的网络架构, 我们将其命名为 *dueling architecture*, 明确地将状态值的表示和 (状态相关的) 动作优势分开。决斗架构由两个代表价值和优势功能的流组成, 同时共享一个共同的

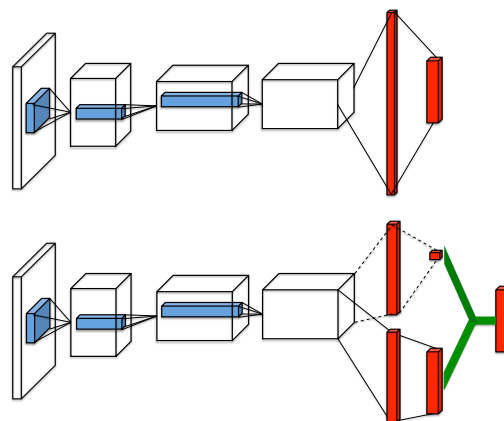


图 1. 流行的单流 Q 网络 (上) 和对抗的 Q 网络 (下)。决斗网络有两个流来分别估计 (标量) 状态值和每个动作的优势; 绿色输出模块实现等式 (9) 以将它们结合起来。两个网络都为每个操作输出 Q 值。

convolutional feature learning module. The two streams are combined via a special aggregating layer to produce an estimate of the state-action value function Q as shown in Figure 1. This dueling network should be understood as a single Q network with two streams that replaces the popular single-stream Q network in existing algorithms such as Deep Q-Networks (DQN; Mnih et al., 2015). The dueling network automatically produces separate estimates of the state value function and advantage function, without any extra supervision.

Intuitively, the dueling architecture can learn which states are (or are not) valuable, without having to learn the effect of each action for each state. This is particularly useful in states where its actions do not affect the environment in any relevant way. To illustrate this, consider the saliency maps shown in Figure 2¹. These maps were generated by computing the Jacobians of the trained value and advantage streams with respect to the input video, following the method proposed by Simonyan et al. (2013). (The experimental section describes this methodology in more detail.) The figure shows the value and advantage saliency maps for two different time steps. In one time step (leftmost pair of images), we see that the value network stream pays attention to the road and in particular to the horizon, where new cars appear. It also pays attention to the score. The advantage stream on the other hand does not pay much attention to the visual input because its action choice is practically irrelevant when there are no cars in front. However, in the second time step (rightmost pair of images) the advantage stream pays attention as there is a car immediately in front, making its choice of action very relevant.

In the experiments, we demonstrate that the dueling architecture can more quickly identify the correct action during policy evaluation as redundant or similar actions are added to the learning problem.

We also evaluate the gains brought in by the dueling architecture on the challenging Atari 2600 testbed. Here, an RL agent with the same structure and hyper-parameters must be able to play 57 different games by observing image pixels and game scores only. The results illustrate vast improvements over the single-stream baselines of Mnih et al. (2015) and van Hasselt et al. (2015). The combination of prioritized replay (Schaul et al., 2016) with the proposed dueling network results in the new state-of-the-art for this popular domain.

1.1. Related Work

The notion of maintaining separate value and advantage functions goes back to Baird (1993). In Baird’s original

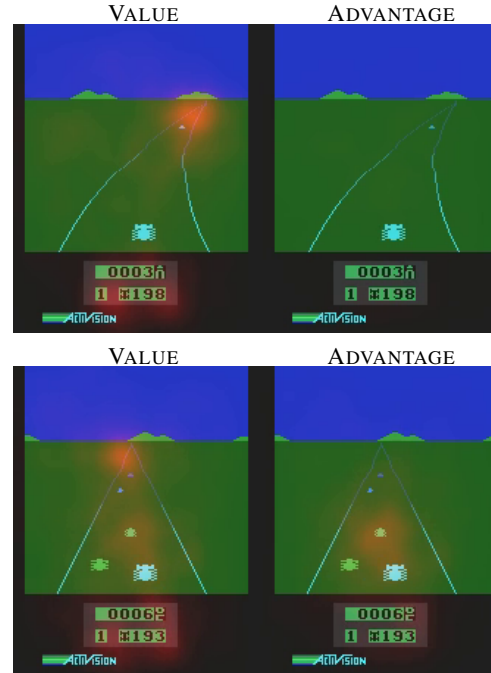


Figure 2. See, attend and drive: Value and advantage saliency maps (red-tinted overlay) on the Atari game Enduro, for a trained dueling architecture. The value stream learns to pay attention to the road. The advantage stream learns to pay attention only when there are cars immediately in front, so as to avoid collisions.

advantage updating algorithm, the shared Bellman residual update equation is decomposed into two updates: one for a state value function, and one for its associated advantage function. Advantage updating was shown to converge faster than Q-learning in simple continuous time domains in (Harmon et al., 1995). Its successor, the advantage learning algorithm, represents only a single advantage function (Harmon & Baird, 1996).

The dueling architecture represents both the value $V(s)$ and advantage $A(s, a)$ functions with a single deep model whose output combines the two to produce a state-action value $Q(s, a)$. Unlike in advantage updating, the representation and algorithm are decoupled by construction. Consequently, the dueling architecture can be used in combination with a myriad of model free RL algorithms.

There is a long history of advantage functions in policy gradients, starting with (Sutton et al., 2000). As a recent example of this line of work, Schulman et al. (2015) estimate advantage values online to reduce the variance of policy gradient algorithms.

There have been several attempts at playing Atari with deep reinforcement learning, including Mnih et al. (2015); Guo et al. (2014); Stadie et al. (2015); Nair et al. (2015); van Hasselt et al. (2015); Bellemare et al. (2016) and Schaul

¹<https://www.youtube.com/playlist?list=PLVFXyCSfS2Pau0gBh0mwTxDmutywWyFBP>

卷积特征学习模块。这两个流通过一个特殊的聚合层组合起来，产生状态-动作值函数 Q 的估计，如图 1 所示。这个决斗网络应该被理解为具有两个流的单个 Q 网络，它取代了现有算法中流行的单流 Q 网络，例如 Deep Q-Networks (DQN; Mnih et al., 2015)。决斗网络自动产生状态价值函数和优势函数的单独估计，无需任何额外的监督。

直观上，决斗架构可以了解哪些状态有价值（或没有价值），而不必了解每个动作对每个状态的影响。这在其行为不会以任何相关方式影响环境的州特别有用。为了说明这一点，请考虑图 2¹ 中所示的显着图。这些图是通过计算训练值和优势流相对于输入视频的雅可比行列式生成的，遵循 Simonyan 等人提出的方法。

（2013）。（实验部分更详细地描述了这种方法。）该图显示了两个不同时间步骤的价值和优势显着性图。在一个时间步长（最左边的一对图像）中，我们看到价值网络流关注道路，特别是新车出现的地平线。它还注重分数。另一方面，优势流并不太关注视觉输入，因为当前面没有车辆时，它的动作选择实际上是无关紧要的。然而，在第二个时间步长（最右边的一对图像）中，优势流会注意到前面有一辆汽车，这使得它的动作选择非常相关。

在实验中，我们证明，当冗余或类似的动作被添加到学习问题中时，决斗架构可以在策略评估期间更快地识别正确的动作。

我们还在具有挑战性的 Atari 2600 测试平台上评估了决斗架构带来的收益。在这里，具有相同结构和超参数的 RL 代理必须能够仅通过观察图像像素和游戏分数来玩 57 种不同的游戏。结果表明，与 Mnih 等人的单流基线相比，有了巨大的改进。（2015）和范哈塞尔特等人。（2015）。优先重放（Schaul 等人，2016）与所提出的决斗网络的结合为这个流行领域带来了新的最先进技术。

1.1.相关工作

维持独立的价值函数和优势函数的概念可以追溯到贝尔德（Baird, 1993）。在贝尔德的原著中

¹<https://www.youtube.com/playlist?list=PLVFXyCSfS2Pau0gBh0mwTxDmutywWyFBP>

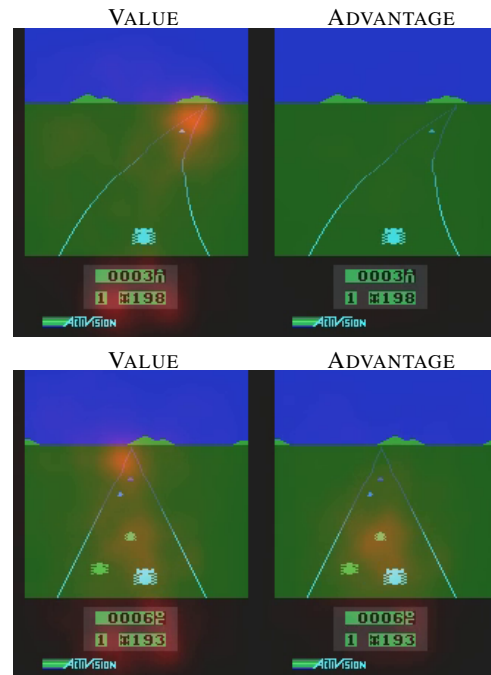


图 2. 查看、参与和驾驶：Atari 游戏 Enduro 上的价值和优势显着性地图（红色叠加），用于训练有素的决斗架构。价值流学会关注道路。优势流学会仅在前方有汽车时才注意，以避免碰撞。

在优势更新算法中，共享贝尔曼残差更新方程被分解为两个更新：一个用于状态值函数，另一个用于其相关的优势函数。在简单的连续时域中，优势更新被证明比 Q 学习收敛得更快（Harmon 等人，1995）。它的后继者，优势学习算法，仅代表单一优势函数（Harmon & Baird, 1996）。

决斗架构用单个深度模型表示价值 $V(s)$ 和优势 $A(s, a)$ 函数，其输出将两者结合起来产生状态动作值 $Q(s, a)$ 。与优势更新不同，表示和算法通过构造解耦。因此，决斗架构可以与多种无模型强化学习算法结合使用。

政策梯度中的优势函数由来已久（Sutton et al., 2000）。作为这方面工作的最新例子，Schulman 等人。（2015）在线估计优势值以减少策略梯度算法的方差。

人们已经多次尝试通过深度强化学习来玩 Atari，其中包括 Mnih 等人。（2015）；郭等人。（2014）；斯塔迪等人。（2015）；奈尔等人。（2015）；范哈塞尔特等人。（2015）；贝尔马尔等人。（2016）和斯考尔

et al. (2016). The results of Schaul et al. (2016) are the current published state-of-the-art.

2. Background

We consider a sequential decision making setup, in which an agent interacts with an environment $\mathcal{E}$ over discrete time steps, see Sutton & Barto (1998) for an introduction. In the Atari domain, for example, the agent perceives a video s_t consisting of M image frames: $s_t = (x_{t-M+1}, \dots, x_t) \in \mathcal{S}$ at time step t . The agent then chooses an action from a discrete set $a_t \in \mathcal{A} = \{1, \dots, |\mathcal{A}|\}$ and observes a reward signal r_t produced by the game emulator.

The agent seeks maximize the expected discounted return, where we define the discounted return as $R_t = \sum_{\tau=t}^{\infty} \gamma^{\tau-t} r_{\tau}$. In this formulation, $\gamma \in [0, 1]$ is a discount factor that trades-off the importance of immediate and future rewards.

For an agent behaving according to a stochastic policy π , the values of the state-action pair (s, a) and the state s are defined as follows

$$\begin{aligned} Q^{\pi}(s, a) &= \mathbb{E}[R_t | s_t = s, a_t = a, \pi], \text{ and} \\ V^{\pi}(s) &= \mathbb{E}_{a \sim \pi(s)}[Q^{\pi}(s, a)]. \end{aligned} \quad (1)$$

The preceding state-action value function (Q function for short) can be computed recursively with dynamic programming:

$$Q^{\pi}(s, a) = \mathbb{E}_{s'}[r + \gamma \mathbb{E}_{a' \sim \pi(s')}[Q^{\pi}(s', a')] | s, a, \pi].$$

We define the optimal $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$. Under the deterministic policy $a = \arg \max_{a' \in \mathcal{A}} Q^*(s, a')$, it follows that $V^*(s) = \max_a Q^*(s, a)$. From this, it also follows that the optimal Q function satisfies the Bellman equation:

$$Q^*(s, a) = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q^*(s', a') | s, a]. \quad (2)$$

We define another important quantity, the *advantage function*, relating the value and Q functions:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s). \quad (3)$$

Note that $\mathbb{E}_{a \sim \pi(s)}[A^{\pi}(s, a)] = 0$. Intuitively, the value function V measures the how good it is to be in a particular state s . The Q function, however, measures the the value of choosing a particular action when in this state. The advantage function subtracts the value of the state from the Q function to obtain a relative measure of the importance of each action.

2.1. Deep Q-networks

The value functions as described in the preceding section are high dimensional objects. To approximate them, we can use a deep Q -network: $Q(s, a; \theta)$ with parameters θ . To estimate this network, we optimize the following sequence of loss functions at iteration i :

$$L_i(\theta_i) = \mathbb{E}_{s, a, r, s'} \left[\left(y_i^{DQN} - Q(s, a; \theta_i) \right)^2 \right], \quad (4)$$

with

$$y_i^{DQN} = r + \gamma \max_{a'} Q(s', a'; \theta^-), \quad (5)$$

where θ^- represents the parameters of a fixed and separate *target network*. We could attempt to use standard Q -learning to learn the parameters of the network $Q(s, a; \theta)$ online. However, this estimator performs poorly in practice. A key innovation in (Mnih et al., 2015) was to freeze the parameters of the target network $Q(s', a'; \theta^-)$ for a fixed number of iterations while updating the *online network* $Q(s, a; \theta_i)$ by gradient descent. (This greatly improves the stability of the algorithm.) The specific gradient update is

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a, r, s'} \left[\left(y_i^{DQN} - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

This approach is model free in the sense that the states and rewards are produced by the environment. It is also off-policy because these states and rewards are obtained with a behavior policy (epsilon greedy in DQN) different from the online policy that is being learned.

Another key ingredient behind the success of DQN is *experience replay* (Lin, 1993; Mnih et al., 2015). During learning, the agent accumulates a dataset $\mathcal{D}_t = \{e_1, e_2, \dots, e_t\}$ of experiences $e_t = (s_t, a_t, r_t, s_{t+1})$ from many episodes. When training the Q -network, instead only using the current experience as prescribed by standard temporal-difference learning, the network is trained by sampling mini-batches of experiences from $\mathcal{D}$ uniformly at random. The sequence of losses thus takes the form

$$L_i(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{U}(\mathcal{D})} \left[\left(y_i^{DQN} - Q(s, a; \theta_i) \right)^2 \right].$$

Experience replay increases data efficiency through re-use of experience samples in multiple updates and, importantly, it reduces variance as uniform sampling from the replay buffer reduces the correlation among the samples used in the update.

2.2. Double Deep Q-networks

The previous section described the main components of DQN as presented in (Mnih et al., 2015). In this paper,

等人。(2016)。Schaul 等人的结果。(2016) 是当前发布的最新技术。

2. 背景

我们考虑一个顺序决策设置，其中代理通过离散时间步与环境 $\mathcal{E}$ 交互，请参阅 Sutton & Barto (1998) 的介绍。例如，在 Atari 域中，代理在时间步 t 感知由 M 个图像帧组成的视频 s_t : $s_t = (x_{t-M+1}, \dots, x_t) \in \mathcal{S}$ 。然后，代理从离散集合 $a_t \in \mathcal{A} = \{1, \dots, |\mathcal{A}|\}$ 中选择一个动作，并观察游戏模拟器产生的奖励信号 r_t 。

代理寻求最大化预期折扣回报，我们将折扣回报定义为 $R_t = \sum_{\tau=t}^{\infty} \gamma^{\tau-t} r_{\tau}$ 。在这个公式中， $\gamma \in [0, 1]$ 是一个折扣因子，用于权衡即时奖励和未来奖励的重要性。

对于根据随机策略 π 行事的代理，状态-动作对 (s, a) 和状态 s 的值定义如下

$$\begin{aligned} Q^{\pi}(s, a) &= \mathbb{E}[R_t | s_t = s, a_t = a, \pi], \text{ and} \\ V^{\pi}(s) &= \mathbb{E}_{a \sim \pi(s)}[Q^{\pi}(s, a)]. \end{aligned} \quad (1)$$

前面的状态动作值函数（简称 Q 函数）可以通过动态规划递归计算：

$$Q^{\pi}(s, a) = \mathbb{E}_{s'}[r + \gamma \mathbb{E}_{a' \sim \pi(s')}[Q^{\pi}(s', a')] | s, a, \pi].$$

我们定义最优 $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$ 。在确定性策略 $a = \arg \max_{a' \in \mathcal{A}} Q^*(s, a')$ 下，可以得出 $V^*(s) = \max_a Q^*(s, a)$ 。由此，还可以得出最优 Q 函数满足贝尔曼方程：

$$Q^*(s, a) = \mathbb{E}_{s'}[r + \gamma \max_{a'} Q^*(s', a') | s, a]. \quad (2)$$

我们定义另一个重要的量，*advantage function*，将值和 Q 函数联系起来：

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s). \quad (3)$$

请注意， $\mathbb{E}_{a \sim \pi(s)}[A^{\pi}(s, a)] = 0$ 。直观上，价值函数 V 衡量处于特定状态 s 的程度。然而， Q 函数测量在此状态下选择特定操作的价值。优势函数从 Q 函数中减去状态值，以获得每个动作重要性的相对度量。

2.1. 深度 Q 网络

上一节中描述的值函数是高维对象。为了近似它们，我们可以使用深度 Q 网络： $Q(s, a; \theta)$ 与参数 θ 。为了估计该网络，我们在迭代 i 时优化以下损失函数序列：

$$L_i(\theta_i) = \mathbb{E}_{s, a, r, s'} \left[\left(y_i^{DQN} - Q(s, a; \theta_i) \right)^2 \right], \quad (4)$$

和

$$y_i^{DQN} = r + \gamma \max_{a'} Q(s', a'; \theta^-), \quad (5)$$

其中 θ^- 表示固定且独立的 *target network* 的参数。我们可以尝试使用标准的 Q -学习来学习网络的参数 $Q(s, a; \theta)$ 在线。然而，该估计器在实践中表现不佳。(Mnih et al., 2015) 的一个关键创新是冻结目标网络 $Q(s', a'; \theta^-)$ 的参数； θ^- 进行固定次数的迭代，同时更新 *online network* $Q(s, a; \theta_i)$ 通过梯度下降。（这大大提高了算法的稳定性。）具体的梯度更新为

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a, r, s'} \left[\left(y_i^{DQN} - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

这种方法是无模型的，因为状态和奖励是由环境产生的。它也是脱策略的，因为这些状态和奖励是通过与正在学习的在线策略不同的行为策略（DQN 中的 *epsilon greedy*）获得的。

DQN 成功背后的另一个关键因素是 *experience replay* (Lin, 1993; Mnih 等人, 2015)。在学习过程中，代理积累了许多情节中的经验 $e_t = (s_t, a_t, r_t, s_{t+1})$ 的数据集 $\mathcal{D}_t = \{e_1, e_2, \dots, e_t\}$ 。在训练 Q 网络时，不是仅使用标准时间差异学习规定的当前经验，而是通过从 $\mathcal{D}$ 均匀随机采样小批量经验来训练网络。因此，损失序列的形式为

$$L_i(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{U}(\mathcal{D})} \left[\left(y_i^{DQN} - Q(s, a; \theta_i) \right)^2 \right].$$

经验重放通过在多次更新中重复使用经验样本来提高数据效率，重要的是，它减少了方差，因为来自重放缓冲区的均匀采样减少了更新中使用的样本之间的相关性。

2.2. 双深 Q 网络

上一节描述了 (Mnih et al., 2015) 中介绍的 DQN 的主要组成部分。在本文中，

we use the improved Double DQN (DDQN) learning algorithm of van Hasselt et al. (2015). In Q-learning and DQN, the max operator uses the same values to both select and evaluate an action. This can therefore lead to overoptimistic value estimates (van Hasselt, 2010). To mitigate this problem, DDQN uses the following target:

$$y_i^{DDQN} = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta_i); \theta^-). \quad (6)$$

DDQN is the same as for DQN (see Mnih et al. (2015)), but with the target y_i^{DDQN} replaced by y_i^{DDQN} . The pseudo-code for DDQN is presented in Appendix A.

2.3. Prioritized Replay

A recent innovation in prioritized experience replay (Schaul et al., 2016) built on top of DDQN and further improved the state-of-the-art. Their key idea was to increase the replay probability of experience tuples that have a high expected learning progress (as measured via the proxy of absolute TD-error). This led to both faster learning and to better final policy quality across most games of the Atari benchmark suite, as compared to uniform experience replay.

To strengthen the claim that our dueling architecture is complementary to algorithmic innovations, we show that it improves performance for both the uniform and the prioritized replay baselines (for which we picked the easier to implement rank-based variant), with the resulting prioritized dueling variant holding the new state-of-the-art.

3. The Dueling Network Architecture

The key insight behind our new architecture, as illustrated in Figure 2, is that for many states, it is unnecessary to estimate the value of each action choice. For example, in the Enduro game setting, knowing whether to move left or right only matters when a collision is eminent. In some states, it is of paramount importance to know which action to take, but in many other states the choice of action has no repercussion on what happens. For bootstrapping based algorithms, however, the estimation of state values is of great importance for every state.

To bring this insight to fruition, we design a single Q -network architecture, as illustrated in Figure 1, which we refer to as the dueling network. The lower layers of the dueling network are convolutional as in the original DQNs (Mnih et al., 2015). However, instead of following the convolutional layers with a single sequence of fully connected layers, we instead use two sequences (or streams) of fully connected layers. The streams are constructed such that they have the capability of providing separate estimates of the value and advantage functions. Finally, the two streams are combined to produce a single output Q

function. As in (Mnih et al., 2015), the output of the network is a set of Q values, one for each action.

Since the output of the dueling network is a Q function, it can be trained with the many existing algorithms, such as DDQN and SARSA. In addition, it can take advantage of any improvements to these algorithms, including better replay memories, better exploration policies, intrinsic motivation, and so on.

The module that combines the two streams of fully-connected layers to output a Q estimate requires very thoughtful design.

From the expressions for advantage $Q^\pi(s, a) = V^\pi(s) + A^\pi(s, a)$ and state-value $V^\pi(s) = \mathbb{E}_{a \sim \pi(s)} [Q^\pi(s, a)]$, it follows that $\mathbb{E}_{a \sim \pi(s)} [A^\pi(s, a)] = 0$. Moreover, for a deterministic policy, $a^* = \arg \max_{a' \in \mathcal{A}} Q(s, a')$, it follows that $Q(s, a^*) = V(s)$ and hence $A(s, a^*) = 0$.

Let us consider the dueling network shown in Figure 1, where we make one stream of fully-connected layers output a scalar $V(s; \theta, \beta)$, and the other stream output an $|\mathcal{A}|$ -dimensional vector $A(s, a; \theta, \alpha)$. Here, θ denotes the parameters of the convolutional layers, while α and β are the parameters of the two streams of fully-connected layers.

Using the definition of advantage, we might be tempted to construct the aggregating module as follows:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha), \quad (7)$$

Note that this expression applies to all (s, a) instances; that is, to express equation (7) in matrix form we need to replicate the scalar, $V(s; \theta, \beta)$, $|\mathcal{A}|$ times.

However, we need to keep in mind that $Q(s, a; \theta, \alpha, \beta)$ is only a parameterized estimate of the true Q -function. Moreover, it would be wrong to conclude that $V(s; \theta, \beta)$ is a good estimator of the state-value function, or likewise that $A(s, a; \theta, \alpha)$ provides a reasonable estimate of the advantage function.

Equation (7) is unidentifiable in the sense that given Q we cannot recover V and A uniquely. To see this, add a constant to $V(s; \theta, \beta)$ and subtract the same constant from $A(s, a; \theta, \alpha)$. This constant cancels out resulting in the same Q value. This lack of identifiability is mirrored by poor practical performance when this equation is used directly.

To address this issue of identifiability, we can force the advantage function estimator to have zero advantage at the chosen action. That is, we let the last module of the network implement the forward mapping

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha) \right). \quad (8)$$

我们使用 van Hasselt 等人改进的 Double DQN (DDQN) 学习算法。(2015)。在 Q 学习和 DQN 中，最大运算符使用相同的值来选择和评估动作。因此，这可能导致过度乐观的价值估计 (van Hasselt, 2010)。为了缓解此问题，DDQN 使用以下目标：

$$y_i^{DDQN} = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta_i); \theta^-). \quad (6)$$

DDQN 与 DQN 相同 (参见 Mnih et al. (2015))，但目标 y_i^{DQN} 替换为 y_i^{DDQN} 。DDQN 的伪代码在附录 A 中给出。

2.3. 优先重播

最近在优先体验重播方面的一项创新 (Schaul 等人, 2016) 建立在 DDQN 之上，并进一步改进了最先进的技术。他们的关键想法是增加具有较高预期学习进度的经验元组的重播概率 (通过绝对 TD 误差的代理来测量)。与统一的体验重放相比，Atari 基准套件的大多数游戏都可以更快地学习并获得更好的最终策略质量。

为了强化我们的决斗架构是对算法创新的补充这一主张，我们证明它提高了统一和优先重放基线 (为此我们选择了更容易实现的基于排名的变体) 的性能，最终的优先决斗变体保持了新的最先进技术。

3. 决斗网络架构

如图 2 所示，我们的新架构背后的关键见解是，对于许多状态来说，没有必要估计每个行动选择的价值。例如，在耐力赛游戏设置中，只有在发生碰撞时才知道是向左还是向右移动才重要。在一些州，知道采取哪种行动至关重要，但在许多其他州，行动的选择对所发生的事情没有影响。然而，对于基于引导的算法，状态值的估计对于每个状态都非常重要。

为了实现这一见解，我们设计了一个单一的 Q- 网络架构，如图 1 所示，我们将其称为决斗网络。决斗网络的较低层与原始 DQN 一样是卷积的 (Mnih 等人, 2015)。然而，我们不是使用单个全连接层序列跟随卷积层，而是使用两个全连接层序列 (或流)。这些流的构造使得它们有能力提供价值函数和优势函数的单独估计。最后，两个流被组合起来产生一个输出 Q

功能。如 (Mnih 等人, 2015) 所示，网络的输出是一组 Q 值，每个动作一个。

由于决斗网络的输出是一个 Q 函数，因此可以使用许多现有算法进行训练，例如 DDQN 和 SARSA。此外，它还可以利用这些算法的任何改进，包括更好的重放记忆、更好的探索策略、内在动机等等。

组合两个全连接层流以输出 Q 估计的模块需要非常周到的设计。

根据优势 $Q^\pi(s, a) = V^\pi(s) + A^\pi(s, a)$ 和状态值 $V^\pi(s) = \mathbb{E}_{a \sim \pi(s)} [Q^\pi(s, a)]$ 的表达式，可以得出 $\mathbb{E}_{a \sim \pi(s)} [A^\pi(s, a)] = 0$ 。此外，对于确定性策略 $a^* = \arg \max_{a' \in \mathcal{A}} Q(s, a')$ ，可以得出 $Q(s, a^*) = V(s)$ 和 $A(s, a^*) = 0$ 。

让我们考虑图 1 中所示的决斗网络，其中我们使全连接层的一个流输出标量 $V(s; \theta, \beta)$ ，另一个流输出一个 $|\mathcal{A}|$ 维向量 $A(s, a; \theta, \alpha)$ 。这里， θ 表示卷积层的参数，而 α 和 β 是全连接层的两个流的参数。

使用优势的估计，我们可能会尝试构建聚合模块，如下所示：

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha), \quad (7)$$

请注意，此表达式适用于所有 (s, a) 实例；也就是说，为了以矩阵形式表达方程 (7)，我们需要复制标量 $V(s; \theta, \beta)$ 、 $|\mathcal{A}|$ 次。

然而，我们需要记住 $Q(s, a; \theta, \alpha, \beta)$ 只是真实 Q 函数的参数化估计。此外，得出 $V(s)$ 的结论也是错误的。 θ, β 是状态值函数的良好估计器，或者同样是 $A(s, a; \theta, \alpha)$ 提供了对优势函数的合理估计。

方程 (7) 是不可辨识的，因为给定 Q，我们无法唯一地恢复 V 和 A。要看到这一点，请向 $V(s)$ 添加一个常量； θ, β 并从 $A(s, a)$ 中减去相同的常数； θ, α 。该常数相互抵消，产生相同的 Q 值。当直接使用该方程时，实际性能不佳反映了可识别性的缺乏。

为了解决这个可识别性问题，我们可以强制优势函数估计器在所选动作上具有零优势。也就是说，我们让网络的最后一个模块实现前向映射

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \max_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha) \right). \quad (8)$$

Now, for $a^* = \arg \max_{a' \in \mathcal{A}} Q(s, a'; \theta, \alpha, \beta) = \arg \max_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha)$, we obtain $Q(s, a^*; \theta, \alpha, \beta) = V(s; \theta, \beta)$. Hence, the stream $V(s; \theta, \beta)$ provides an estimate of the value function, while the other stream produces an estimate of the advantage function.

An alternative module replaces the max operator with an average:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right). \quad (9)$$

On the one hand this loses the original semantics of V and A because they are now off-target by a constant, but on the other hand it increases the stability of the optimization: with (9) the advantages only need to change as fast as the mean, instead of having to compensate any change to the optimal action's advantage in (8). We also experimented with a softmax version of equation (8), but found it to deliver similar results to the simpler module of equation (9). Hence, all the experiments reported in this paper use the module of equation (9).

Note that while subtracting the mean in equation (9) helps with identifiability, it does not change the relative rank of the A (and hence Q) values, preserving any greedy or ϵ -greedy policy based on Q values from equation (7). When acting, it suffices to evaluate the advantage stream to make decisions.

It is important to note that equation (9) is viewed and implemented as part of the network and not as a separate algorithmic step. Training of the dueling architectures, as with standard Q networks (e.g. the deep Q -network of Mnih et al. (2015)), requires only back-propagation. The estimates $V(s; \theta, \beta)$ and $A(s, a; \theta, \alpha)$ are computed automatically without any extra supervision or algorithmic modifications.

As the dueling architecture shares the same input-output interface with standard Q networks, we can recycle all learning algorithms with Q networks (e.g., DDQN and SARSA) to train the dueling architecture.

4. Experiments

We now show the practical performance of the dueling network. We start with a simple policy evaluation task and then show larger scale results for learning policies for general Atari game-playing.

4.1. Policy evaluation

We start by measuring the performance of the dueling architecture on a policy evaluation task. We choose this par-

ticular task because it is very useful for evaluating network architectures, as it is devoid of confounding factors such as the choice of exploration strategy, and the interaction between policy improvement and policy evaluation.

In this experiment, we employ temporal difference learning (without eligibility traces, i.e., $\lambda = 0$) to learn Q values. More specifically, given a behavior policy π , we seek to estimate the state-action value $Q^\pi(\cdot, \cdot)$ by optimizing the sequence of costs of equation (4), with target

$$y_i = r + \gamma \mathbb{E}_{a' \sim \pi(s')} [Q(s', a'; \theta_i)].$$

The above update rule is the same as that of Expected SARSA (van Seijen et al., 2009). We, however, do not modify the behavior policy as in Expected SARSA.

To evaluate the learned Q values, we choose a simple environment where the exact $Q^\pi(s, a)$ values can be computed separately for all $(s, a) \in \mathcal{S} \times \mathcal{A}$. This environment, which we call the *corridor* is composed of three connected corridors. A schematic drawing of the corridor environment is shown in Figure 3. The agent starts from the bottom left corner of the environment and must move to the top right to get the largest reward. A total of 5 actions are available: go up, down, left, right and no-op. We also have the freedom of adding an arbitrary number of no-op actions. In our setup, the two vertical sections both have 10 states while the horizontal section has 50.

We use an ϵ -greedy policy as the behavior policy π , which chooses a random action with probability ϵ or an action according to the optimal Q function $\arg \max_{a \in \mathcal{A}} Q^*(s, a)$ with probability $1 - \epsilon$. In our experiments, ϵ is chosen to be 0.001.

We compare a single-stream Q architecture with the dueling architecture on three variants of the corridor environment with 5, 10 and 20 actions respectively. The 10 and 20 action variants are formed by adding no-ops to the original environment. We measure performance by Squared Error (SE) against the true state values: $\sum_{s \in \mathcal{S}, a \in \mathcal{A}} (Q(s, a; \theta) - Q^\pi(s, a))^2$. The single-stream architecture is a three layer MLP with 50 units on each hidden layer. The dueling architecture is also composed of three layers. After the first hidden layer of 50 units, however, the network branches off into two streams each of them a two layer MLP with 25 hidden units. The results of the comparison are summarized in Figure 3.

The results show that with 5 actions, both architectures converge at about the same speed. However, when we increase the number of actions, the dueling architecture performs better than the traditional Q -network. In the dueling network, the stream $V(s; \theta, \beta)$ learns a general value that is shared across many similar actions at s , hence leading to faster convergence. This is a very promising result be-

现在，对于 $a^* = \arg \max_{a' \in \mathcal{A}} Q(s, a'; \theta, \alpha, \beta) = \arg \max_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha)$ ，我们得到 $Q(s, a^*; \theta, \alpha, \beta) = V(s; \theta, \beta)$ 。因此，流 $V(s; \theta, \beta)$ 提供价值函数的估计，而另一个流产生优势函数的估计。

另一个模块用平均值替换最大运算符：

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right). \quad (9)$$

一方面，这失去了 V 和 A 的原始语义，因为它们现在偏离目标是一个常量，但另一方面，它增加了优化的稳定性：使用 (9)，优势只需要像均值一样快地改变，而不需要补偿 (8) 中最优动作优势的任何变化。我们还尝试了方程 (8) 的 Softmax 版本，但发现它可以提供与方程 (9) 的更简单模块类似的结果。因此，本文报道的所有实验均使用方程 (9) 的模块。

请注意，虽然减去等式 (9) 中的平均值有助于提高可识别性，但它不会改变 A (的相对排名，因此不会改变 Q) 值，从而保留基于等式 (7) 中的 Q 值的任何贪婪或 ϵ -贪婪策略。行动时，评估优势流就足以做出决策。

值得注意的是，方程 (9) 被视为和实现为网络的一部分，而不是作为单独的算法步骤。与标准 Q 网络（例如 Mnih 等人 (2015) 的深度 Q 网络）一样，对决架构的训练仅需要反向传播。估计 $V(s; \theta, \beta)$ 和 $A(s, a; \theta, \alpha)$ 是自动计算的，无需任何额外的监督或算法修改。

由于决斗架构与标准 Q 网络共享相同的输入输出接口，因此我们可以利用 Q 网络（e.g.、DDQN 和 SARSA）回收所有学习算法来训练决斗架构。

4. 实验

我们现在展示决斗网络的实际性能。我们从一个简单的策略评估任务开始，然后展示用于一般 Atari 游戏的学习策略的更大规模的结果。

4.1. 政策评估

我们首先测量决斗架构在策略评估任务上的性能。我们选择这个参数

这是一个特殊的任务，因为它对于评估网络架构非常有用，因为它没有诸如探索策略的选择以及策略改进和策略评估之间的相互作用等混杂因素。

在这个实验中，我们采用时间差异学习（没有资格痕迹，i.e., $\lambda = 0$ ）来学习 Q 值。更具体地说，给定一个行为策略 π ，我们试图通过优化方程 (4) 的成本序列来估计状态动作值 $Q^\pi(\cdot, \cdot)$ ，目标是

$$y_i = r + \gamma \mathbb{E}_{a' \sim \pi(s')} [Q(s', a'; \theta_i)].$$

上述更新规则与预期 SARSA 相同 (van Seijen et al., 2009)。然而，我们不会像预期的 SARSA 那样修改行为政策。

为了评估学习到的 Q 值，我们选择一个简单的环境，可以为所有 $(s, a) \in \mathcal{S} \times \mathcal{A}$ 单独计算精确的 $Q^\pi(s, a)$ 值。这个环境，我们称之为 *corridor*，由三个相连的走廊组成。走廊环境示意图如图3所示，智能体从环境的左下角开始，必须移动到右上角才能获得最大的奖励。共有 5 种可用操作：上、下、左、右和无操作。我们还可以自由添加任意数量的无操作操作。在我们的设置中，两个垂直部分都有 10 个状态，而水平部分有 50 个状态。

我们使用 ϵ -贪婪策略作为行为策略 π ，它以概率 ϵ 选择随机动作，或者根据最优 Q 函数 $\arg \max_{a \in \mathcal{A}} Q^*(s, a)$ 以概率 $1 - \epsilon$ 选择动作。在我们的实验中， ϵ 被选择为 0.001。

我们在分别有 5、10 和 20 个动作的走廊环境的三种变体上将单流 Q 架构与决斗架构进行比较。10 和 20 动作变体是通过向原始环境添加无操作而形成的。我们通过平方误差 (SE) 相对于真实状态值来衡量性能： $\sum_{s \in \mathcal{S}, a \in \mathcal{A}} (Q(s, a; \theta) - Q^\pi(s, a))^2$ 。单流架构是一个三层 MLP，每个隐藏层有 50 个单元。决斗架构也由三层组成。然而，在第一个包含 50 个单元的隐藏层之后，网络分支成两个流，每个流都是具有 25 个隐藏单元的两层 MLP。比较结果总结于图 3 中。

结果表明，通过 5 个操作，两种架构以大致相同的速度收敛。然而，当我们增加动作数量时，决斗架构的性能比传统的 Q 网络更好。在决斗网络中，流 $V(s; \theta, \beta)$ 学习在 s 处的许多类似操作之间共享的通用值，从而实现更快的收敛。这是一个非常有希望的结果

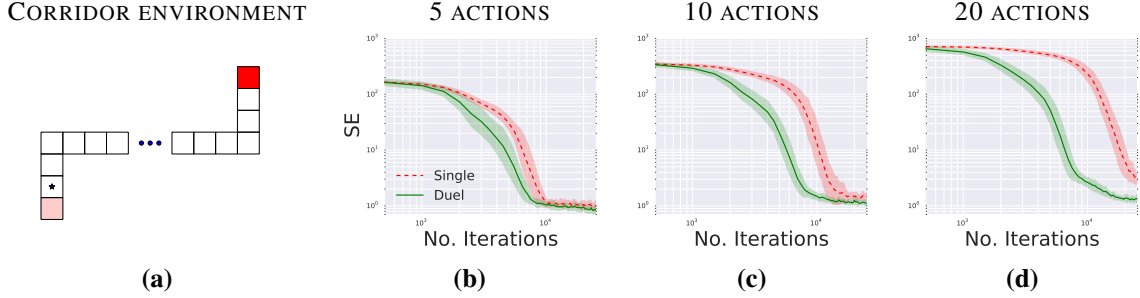


Figure 3. (a) The corridor environment. The star marks the starting state. The redness of a state signifies the reward the agent receives upon arrival. The game terminates upon reaching either reward state. The agent’s actions are going up, down, left, right and no action. Plots (b), (c) and (d) shows squared error for policy evaluation with 5, 10, and 20 actions on a log-log scale. The dueling network (Duel) consistently outperforms a conventional single-stream network (Single), with the performance gap increasing with the number of actions.

cause many control tasks with large action spaces have this property, and consequently we should expect that the dueling network will often lead to much faster convergence than a traditional single stream network. In the following section, we will indeed see that the dueling network results in substantial gains in performance in a wide-range of Atari games.

4.2. General Atari Game-Playing

We perform a comprehensive evaluation of our proposed method on the Arcade Learning Environment (Bellemare et al., 2013), which is composed of 57 Atari games. The challenge is to deploy a single algorithm and architecture, with a fixed set of hyper-parameters, to learn to play all the games given only raw pixel observations and game rewards. This environment is very demanding because it is both comprised of a large number of highly diverse games and the observations are high-dimensional.

We follow closely the setup of van Hasselt et al. (2015) and compare to their results using single-stream Q -networks. We train the dueling network with the DDQN algorithm as presented in Appendix A. At the end of this section, we incorporate prioritized experience replay (Schaul et al., 2016).

Our network architecture has the same low-level convolutional structure of DQN (Mnih et al., 2015; van Hasselt et al., 2015). There are 3 convolutional layers followed by 2 fully-connected layers. The first convolutional layer has $32\ 8 \times 8$ filters with stride 4, the second $64\ 4 \times 4$ filters with stride 2, and the third and final convolutional layer consists $64\ 3 \times 3$ filters with stride 1. As shown in Figure 1, the dueling network splits into two streams of fully connected layers. The value and advantage streams both have a fully-connected layer with 512 units. The final hidden layers of the value and advantage streams are both fully-connected with the value stream having one output and the advantage

as many outputs as there are valid actions². We combine the value and advantage streams using the module described by Equation (9). Rectifier non-linearities (Fukushima, 1980) are inserted between all adjacent layers.

We adopt the optimizers and hyper-parameters of van Hasselt et al. (2015), with the exception of the learning rate which we chose to be slightly lower (we do not do this for double DQN as it can deteriorate its performance). Since both the advantage and the value stream propagate gradients to the last convolutional layer in the backward pass, we rescale the combined gradient entering the last convolutional layer by $1/\sqrt{2}$. This simple heuristic mildly increases stability. In addition, we clip the gradients to have their norm less than or equal to 10. This clipping is not standard practice in deep RL, but common in recurrent network training (Bengio et al., 2013).

To isolate the contributions of the dueling architecture, we re-train DDQN with a single stream network using exactly the same procedure as described above. Specifically, we apply gradient clipping, and use 1024 hidden units for the first fully-connected layer of the network so that both architectures (dueling and single) have roughly the same number of parameters. We refer to this re-trained model as *Single Clip*, while the original trained model of van Hasselt et al. (2015) is referred to as *Single*.

As in (van Hasselt et al., 2015), we start the game with up to 30 *no-op* actions to provide random starting positions for the agent. To evaluate our approach, we measure improvement in percentage (positive or negative) in score over the better of human and baseline agent scores:

$$\frac{\text{Score}_{\text{Agent}} - \text{Score}_{\text{Baseline}}}{\max\{\text{Score}_{\text{Human}}, \text{Score}_{\text{Baseline}}\} - \text{Score}_{\text{Random}}} \cdot \quad (10)$$

We took the maximum over human and baseline agent scores as it prevents insignificant changes to appear as

²The number of actions ranges between 3-18 actions in the ALE environment.

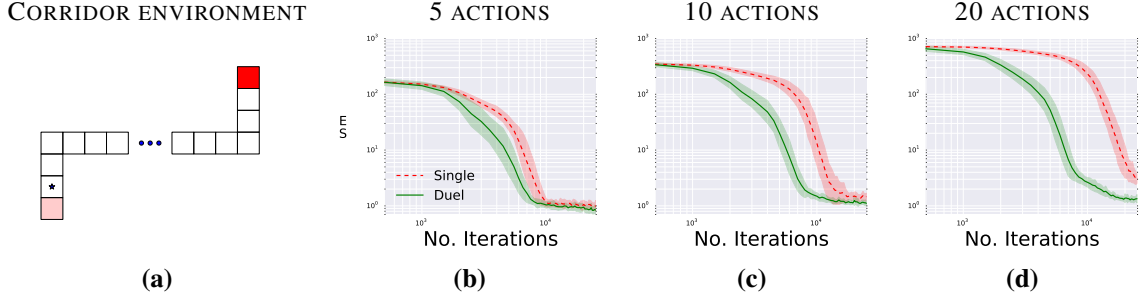


图 3.(a) 走廊环境。星号标志着起始状态。状态的红色表示代理在到达时收到的奖励。游戏在达到任一奖励状态时终止。代理的动作有上、下、左、右和无动作。图 (b)、(c) 和 (d) 显示了对数-对数尺度上 5、10 和 20 个操作的策略评估的平方误差。决斗网络 (Duel) 始终优于传统的单流网络 (Single)，性能差距随着动作数量的增加而增加。

因为许多具有大动作空间的控制任务都具有这种特性，因此我们应该期望决斗网络通常会比传统的单流网络带来更快的收敛速度。在接下来的部分中，我们确实会看到决斗网络在各种 Atari 游戏中带来性能的大幅提升。

4.2. 一般 Atari 游戏

我们在 Arcade 学习环境 (Bellemare 等人, 2013) 上对我们提出的方法进行了全面评估，该环境由 57 个 Atari 游戏组成。挑战在于部署单一的算法和架构，以及一组固定的超参数，以学习在仅给出原始像素观察和游戏奖励的情况下玩所有游戏。这种环境要求非常高，因为它由大量高度多样化的游戏组成，而且观察结果也是高维的。

我们密切关注 van Hasselt 等人的设置 (2015) 并与使用单流 Q 网络的结果进行比较。我们使用附录 A 中介绍的 DDQN 算法训练决斗网络。在本节末尾，我们结合了优先经验回放 (Schaul 等人, 2016)。

我们的网络架构具有与 DQN 相同的低级卷积结构 (Mnih 等人, 2015; van Hasselt 等人, 2015)。有 3 个卷积层，后面是 2 个全连接层。第一个卷积层有 $32 \times 8 \times 8$ 个滤波器，步幅为 4，第二个卷积层有 $64 \times 4 \times 4$ 个滤波器，步幅为 2，第三个也是最后一个卷积层由 $64 \times 3 \times 3$ 个滤波器组成，步幅为 1。如图 1 所示，决斗网络分为两个全连接层流。价值流和优势流都有一个包含 512 个单元的全连接层。价值流和优势流的最终隐藏层都与具有一个输出和优势的价值流完全连接

有多少个有效动作就有多少个输出²。我们使用等式 (9) 描述的模块将价值流和优势流结合起来。整流器非线性 (Fukushima, 1980) 被插入到所有相邻层之间。

我们采用 van Hasselt 等人的优化器和超参数 (2015)，除了我们选择稍低的学习率 (我们不会对双 DQN 这样做，因为它会降低其性能)。由于优势流和价值流都在向后传递中将梯度传播到最后一个卷积层，因此我们将进入最后一个卷积层的组合梯度重新调整为 $1/\sqrt{2}$ 。这种简单的启发式方法稍微提高了稳定性。此外，我们对梯度进行裁剪，使其范数小于或等于 10。这种裁剪不是深度 RL 中的标准做法，但在循环网络训练中很常见 (Bengio 等人, 2013)。

为了隔离决斗架构的贡献，我们使用与上述完全相同的过程使用单流网络重新训练 DDQN。具体来说，我们应用梯度裁剪，并为网络的第一个全连接层使用 10 24 个隐藏单元，以便两种架构 (决斗和单一) 具有大致相同数量的参数。我们将这个重新训练的模型称为 *Single Clip*，而 van Hasselt 等人的原始训练模型 (2015) 被称为 *Single*。

如 (van Hasselt et al., 2015) 中所示，我们以最多 30 *no-op* 个动作开始游戏，为代理提供随机起始位置。为了评估我们的方法，我们测量了相对于人类和基线代理分数中更好的分数的百分比 (正数或负数) 的改进：

$$\frac{\text{Score}_{\text{Agent}} - \text{Score}_{\text{Baseline}}}{\max\{\text{Score}_{\text{Human}}, \text{Score}_{\text{Baseline}}\} - \text{Score}_{\text{Random}}} \quad (10)$$

我们采用了人类和基线代理分数的最大值，因为它可以防止出现微不足道的变化

²The number of actions ranges between 3-18 actions in the ALE environment.

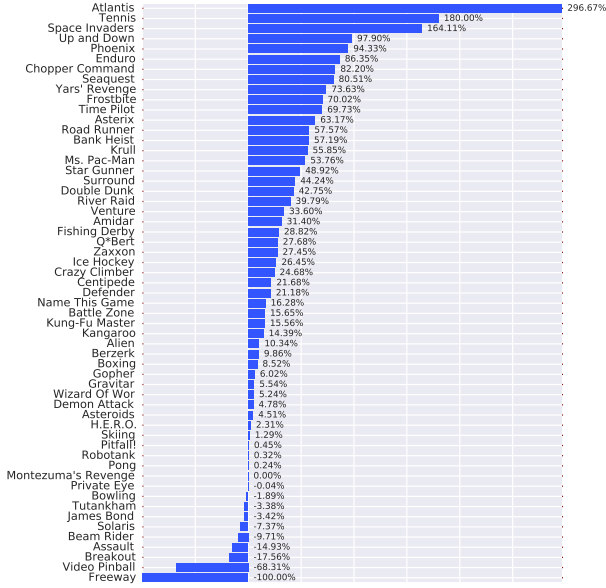


Figure 4. Improvements of dueling architecture over the baseline Single network of van Hasselt et al. (2015), using the metric described in Equation (10). Bars to the right indicate by how much the dueling network outperforms the single-stream network.

large improvements when neither the agent in question nor the baseline are doing well. For example, an agent that achieves 2% human performance should not be interpreted as two times better when the baseline agent achieves 1% human performance. We also chose not to measure performance in terms of percentage of human performance alone because a tiny difference relative to the baseline on some games can translate into hundreds of percent in human performance difference.

The results for the wide suite of 57 games are summarized in Table 1. Detailed results are presented in the Appendix.

Using this 30 no-ops performance measure, it is clear that the dueling network (Duel Clip) does substantially better than the Single Clip network of similar capacity. It also does considerably better than the baseline (Single) of van Hasselt et al. (2015). For comparison we also show results for the deep Q -network of Mnih et al. (2015), referred to as Nature DQN.

Figure 4 shows the improvement of the dueling network over the baseline Single network of van Hasselt et al. (2015). Again, we seen that the improvements are often very dramatic.

As shown in Table 1, Single Clip performs better than Single. We verified that this gain was mostly brought in by gradient clipping. For this reason, we incorporate gradient clipping in all the new approaches.

Table 1. Mean and median scores across all 57 Atari games, measured in percentages of human performance.

	30 no-ops		Human Starts	
	Mean	Median	Mean	Median
Prior. Duel Clip	591.9%	172.1%	567.0%	115.3%
Prior. Single	434.6%	123.7%	386.7%	112.9%
Duel Clip	373.1%	151.5%	343.8%	117.1%
Single Clip	341.2%	132.6%	302.8%	114.1%
Single	307.3%	117.8%	332.9%	110.9%
Nature DQN	227.9%	79.1%	219.6%	68.5%

Duel Clip does better than Single Clip on 75.4% of the games (43 out of 57). It also achieves higher scores compared to the Single baseline on 80.7% (46 out of 57) of the games. Of all the games with 18 actions, Duel Clip is better 86.6% of the time (26 out of 30). This is consistent with the findings of the previous section. Overall, our agent (Duel Clip) achieves human level performance on 42 out of 57 games. Raw scores for all the games, as well as measurements in human performance percentage, are presented in the Appendix.

Robustness to human starts. One shortcoming of the 30 no-ops metric is that an agent does not necessarily have to generalize well to play the Atari games. Due to the deterministic nature of the Atari environment, from a unique starting point, an agent could learn to achieve good performance by simply remembering sequences of actions.

To obtain a more robust measure, we adopt the methodology of Nair et al. (2015). Specifically, for each game, we use 100 starting points sampled from a human expert’s trajectory. From each of these points, an evaluation episode is launched for up to 108,000 frames. The agents are evaluated only on rewards accrued after the starting point. We refer to this metric as *Human Starts*.

As shown in Table 1, under the Human Starts metric, Duel Clip once again outperforms the single stream variants. In particular, our agent does better than the Single baseline on 70.2% (40 out of 57) games and on games of 18 actions, Duel Clip is 83.3% better (25 out of 30).

Combining with Prioritized Experience Replay. The dueling architecture can be easily combined with other algorithmic improvements. In particular, prioritization of the experience replay has been shown to significantly improve performance of Atari games (Schaul et al., 2016). Furthermore, as prioritization and the dueling architecture address very different aspects of the learning process, their combination is promising. So in our final experiment, we investigate the integration of the dueling architecture with prioritized experience replay. We use the prioritized variant of DDQN (Prior. Single) as the new baseline algorithm, which replaces with the uniform sampling of the experi-

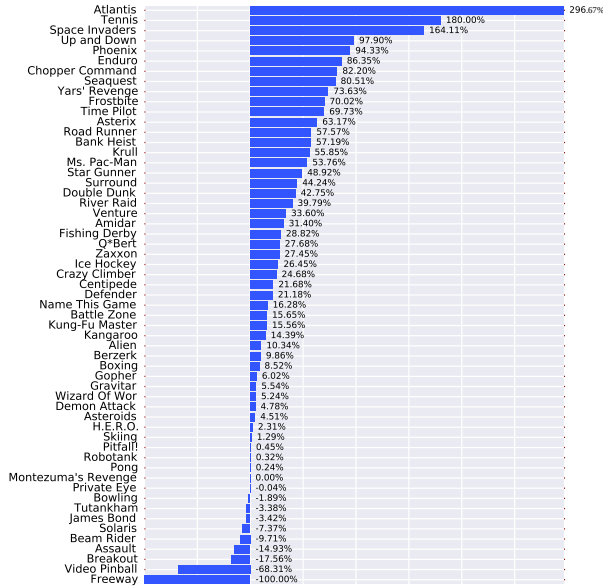


图 4. 决斗架构相对于 van Hasselt 等人的基线单一网络的改进。（2015），使用等式（10）中描述的度量。右侧的条形表示决斗网络优于单流网络的程度。

当相关代理和基线都表现不佳时，会有很大的改进。例如，当基准代理达到人类绩效 1% 时，达到 2% 人类绩效的代理不应被解释为两倍。我们还选择不单独用人类表现的百分比来衡量表现，因为某些游戏相对于基线的微小差异可能会转化为人类表现数百%的差异。

表 1 总结了 57 场比赛的结果。详细结果列于附录中。

使用此 30 个无操作性能测量，很明显，决斗网络 (Duel Clip) 明显优于类似容量的单剪辑网络。它也比 van Hasselt 等人的基线 (Single) 好得多。（2015）。为了进行比较，我们还展示了 Mnih 等人的深度 Q 网络的结果。（2015），简称 Nature DQN。

图 4 显示了 Dueling 网络相对于 van Hasselt 等人的基线 Single 网络的改进。（2015）。我们再次发现，改进往往非常显着。

如表 1 所示，单剪辑的性能优于单剪辑。我们验证了这种增益主要是由梯度裁剪带来的。因此，我们在所有新方法中都加入了梯度裁剪。

表 1. 所有 57 款 Atari 游戏的平均分和中位数分数，以人类表现的百分比来衡量。

	30 no-ops		Human Starts	
	Mean	Median	Mean	Median
Prior. Duel Clip	591.9%	172.1%	567.0%	115.3%
Prior. Single	434.6%	123.7%	386.7%	112.9%
Duel Clip	373.1%	151.5%	343.8%	117.1%
Single Clip	341.2%	132.6%	302.8%	114.1%
Single	307.3%	117.8%	332.9%	110.9%
Nature DQN	227.9%	79.1%	219.6%	68.5%

在 75.4% 的游戏中，Duel Clip 的表现优于 Single Clip（57 场比赛中的 43 场）。与单一基准相比，它在 80.7% 的游戏（57 场中的 46 场）中取得了更高的分数。在所有有 18 个动作的游戏中，Duel Clip 的得分为 86.6%（30 次中的 26 次）。这与上一节的发现是一致的。总体而言，我们的代理 (Duel Clip) 在 57 场比赛中的 42 场比赛中达到了人类水平的表现。附录中列出了所有游戏的原始分数以及人类表现百分比的测量结果。

对人类的鲁棒性开始。30 个无操作指标的一个缺点是，代理不一定需要很好地概括才能玩 Atari 游戏。由于 Atari 环境的不确定性本质，从一个独特的起点开始，智能体可以通过简单地记住动作序列来学习获得良好的性能。

为了获得更稳健的衡量标准，我们采用了 Nair 等人的方法。（2015）。具体来说，对于每个游戏，我们使用从人类专家的轨迹中采样的 100 个起点。从每个点出发，都会启动最多 108,000 帧的评估片段。仅根据起点后累积的奖励来评估代理。我们将此指标称为 *Human Starts*。

如表 1 所示，在 Human Starts 指标下，Duel Clip 再次优于单流变体。特别是，我们的代理在 70.2%（57 场中的 40 场）比赛中比单一基线表现更好，而在 18 个动作的比赛中，Duel Clip 则好 83.3%（30 场中的 25 场）。

与优先体验重放相结合。决斗架构可以轻松地与其他算法改进相结合。特别是，经验回放的优先级已被证明可以显著提高 Atari 游戏的性能（Schaul 等人，2016）。此外，由于优先级和决斗架构解决了学习过程中非常不同的方面，因此它们的组合是有前途的。因此，在我们的最终实验中，我们研究了决斗架构与优先体验重播的集成。我们使用 DDQN (Prior.Single) 的优先变体作为新的基线算法，它取代了经验的均匀采样

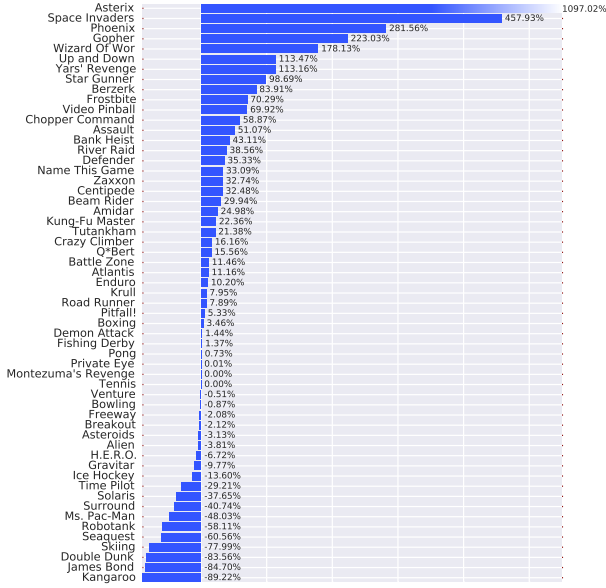


Figure 5. Improvements of dueling architecture over Prioritized DDQN baseline, using the same metric as Figure 4. Again, the dueling architecture leads to significant improvements over the single-stream baseline on the majority of games.

ence tuples by rank-based prioritized sampling. We keep all the parameters of the prioritized replay as described in (Schaul et al., 2016), namely a priority exponent of 0.7, and an annealing schedule on the importance sampling exponent from 0.5 to 1. We combine this baseline with our dueling architecture (as above), and again use gradient clipping (Prior. Duel Clip).

Note that, although orthogonal in their objectives, these extensions (prioritization, dueling and gradient clipping) interact in subtle ways. For example, prioritization interacts with gradient clipping, as sampling transitions with high absolute TD-errors more often leads to gradients with higher norms. To avoid adverse interactions, we roughly re-tuned the learning rate and the gradient clipping norm on a subset of 9 games. As a result of rough tuning, we settled on 6.25×10^{-5} for the learning rate and 10 for the gradient clipping norm (the same as in the previous section).

When evaluated on all 57 Atari games, our prioritized dueling agent performs significantly better than both the prioritized baseline agent and the dueling agent alone. The full mean and median performance against the human performance percentage is shown in Table 1. When initializing the games using up to 30 no-ops action, we observe mean and median scores of 591% and 172% respectively. The direct comparison between the prioritized baseline and prioritized dueling versions, using the metric described in Equation 10, is presented in Figure 5.

The combination of prioritized replay and the dueling network results in vast improvements over the previous state-of-the-art in the popular ALE benchmark.

Saliency maps. To better understand the roles of the value and the advantage streams, we compute saliency maps (Simonyan et al., 2013). More specifically, to visualize the salient part of the image as seen by the value stream, we compute the absolute value of the Jacobian of $\hat{V}$ with respect to the input frames: $|\nabla_s \hat{V}(s; \theta)|$. Similarly, to visualize the salient part of the image as seen by the advantage stream, we compute $|\nabla_s \hat{A}(s, \arg \max_{a'} \hat{A}(s, a'; \theta))|$. Both quantities are of the same dimensionality as the input frames and therefore can be visualized easily alongside the input frames.

Here, we place the gray scale input frames in the green and blue channel and the saliency maps in the red channel. All three channels together form an RGB image. Figure 2 depicts the value and advantage saliency maps on the Enduro game for two different time steps. As observed in the introduction, the value stream pays attention to the horizon where the appearance of a car could affect future performance. The value stream also pays attention to the score. The advantage stream, on the other hand, cares more about cars that are on an immediate collision course.

5. Discussion

The advantage of the dueling architecture lies partly in its ability to learn the state-value function efficiently. With every update of the Q values in the dueling architecture, the value stream V is updated – this contrasts with the updates in a single-stream architecture where only the value for one of the actions is updated, the values for all other actions remain untouched. This more frequent updating of the value stream in our approach allocates more resources to V , and thus allows for better approximation of the state values, which in turn need to be accurate for temporal-difference-based methods like Q-learning to work (Sutton & Barto, 1998). This phenomenon is reflected in the experiments, where the advantage of the dueling architecture over single-stream Q networks grows when the number of actions is large.

Furthermore, the differences between Q -values for a given state are often very small relative to the magnitude of Q . For example, after training with DDQN on the game of Seaquest, the average action gap (the gap between the Q values of the best and the second best action in a given state) across visited states is roughly 0.04, whereas the average state value across those states is about 15. This difference in scales can lead to small amounts of noise in the updates can lead to reorderings of the actions, and thus make the nearly greedy policy switch abruptly. The dueling ar-

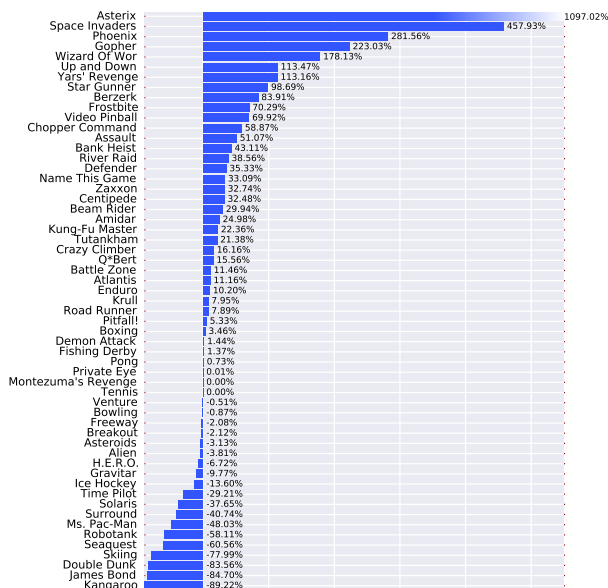


图 5. 决斗架构相对于优先 DDQN 基线的改进，使用与图 4 相同的指标。同样，决斗架构在大多数游戏中比单流基线带来了显着改进。

通过基于排名的优先采样来提取元组。我们保留（Schaul 等人，2016）中描述的优先重放的所有参数，即优先级指数 0.7，以及重要性采样指数从 0.5 到 1 的退火计划。我们将此基线与我们的决斗架构（如上所述）结合起来，并再次使用梯度裁剪（Prior.Duel Clip）。

请注意，虽然它们的目标是正交的，但这些扩展（优先级、决斗和梯度裁剪）以微妙的方式相互作用。例如，优先级与梯度裁剪相互作用，因为具有高绝对 TD 误差的采样转换通常会导致具有更高范数的梯度。为了避免不利的相互作用，我们粗略地重新调整了 9 个游戏子集的学习率和梯度裁剪范数。经过粗调，我们确定学习率为 6.25×10^{-5} ，梯度裁剪范数为 10（与上一节相同）。

当对所有 57 款 Atari 游戏进行评估时，我们的优先决斗代理的表现明显优于优先基线代理和单独的决斗代理。表 1 显示了相对于人类表现百分比的完整平均和中值表现。当使用最多 30 个无操作动作初始化游戏时，我们观察到平均和中值分数分别为 591% 和 172%。使用公式 10 中描述的指标，优先化基线和优先化决斗版本之间的直接比较如图 5 所示。

优先重放和决斗网络的结合使得流行的 ALE 基准测试中以前最先进的技术有了巨大的改进。

显着性图。为了更好地理解价值流和优势流的作用，我们计算显着图（Simonyan et al., 2013）。更具体地说，为了可视化价值流所看到的图像的显着部分，我们计算 $\hat{V}$ 相对于输入帧的雅可比行列式的绝对值： $|\nabla_s \hat{V}(s; \theta)|$ 。类似地，为了可视化优势流所看到的图像的显着部分，我们计算 $|\nabla_s \hat{A}(s, \arg \max_{a'} \hat{A}(s, a'); \theta)|$ 。这两个量与输入帧具有相同的维度，因此可以轻松地与输入帧一起可视化。

在这里，我们将灰度输入帧放置在绿色和蓝色通道中，将显着图放置在红色通道中。所有三个通道一起形成 RGB 图像。图 2 描绘了 Enduro 游戏中两个不同时间步长的价值和优势显着性图。正如引言中所观察到的，价值流关注汽车外观可能影响未来性能的范围。价值流也关注分数。另一方面，优势流更关心立即发生碰撞的汽车。

5. 讨论

决斗架构的优势部分在于其有效学习状态值函数的能力。随着决斗架构中 Q 值的每次更新，价值流 V 都会更新 - 这与单流架构中的更新形成鲜明对比，在单流架构中，仅更新其中一个操作的值，所有其他操作的值保持不变。在我们的方法中，这种更频繁的价值流更新为 V 分配了更多的资源，从而可以更好地逼近状态值，而状态值又需要准确，以便基于时间差异的方法（如 Q-learning）发挥作用（Sutton 和 Barto, 1998）。这种现象在实验中得到了体现，当动作数量很大时，决斗架构相对于单流 Q 网络的优势就会增强。

此外，给定状态的 Q 值之间的差异相对于 Q 的幅度通常非常小。例如，在 Seaquest 游戏上使用 DDQN 进行训练后，所访问状态之间的平均动作间隙（给定状态中最佳动作和次佳动作的 Q 值之间的间隙）大约为 0.04，而这些状态之间的平均状态值约为 15。这种尺度上的差异可能会导致更新中出现少量噪声，从而导致动作重新排序，从而使近乎贪婪的动作变得接近贪婪。政策突然转变。决斗 ar-

chitecture with its separate advantage stream is robust to such effects.

6. Conclusions

We introduced a new neural network architecture that decouples value and advantage in deep Q -networks, while sharing a common feature learning module. The new dueling architecture, in combination with some algorithmic improvements, leads to dramatic improvements over existing approaches for deep RL in the challenging Atari domain. The results presented in this paper are the new state-of-the-art in this popular domain.

References

- Ba, J., Mnih, V., and Kavukcuoglu, K. Multiple object recognition with visual attention. In *ICLR*, 2015.
- Baird, L.C. Advantage updating. Technical Report WL-TR-93-1146, Wright-Patterson Air Force Base, 1993.
- Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
- Bellemare, M. G., Ostrovski, G., Guez, A., Thomas, P. S., and Munos, R. Increasing the action gap: New operators for reinforcement learning. In *AAAI*, 2016. To appear.
- Bengio, Y., Boulanger-Lewandowski, N., and Pascanu, R. Advances in optimizing recurrent networks. In *ICASSP*, pp. 8624–8628, 2013.
- Fukushima, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36:193–202, 1980.
- Guo, X., Singh, S., Lee, H., Lewis, R. L., and Wang, X. Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning. In *NIPS*, pp. 3338–3346, 2014.
- Harmon, M.E. and Baird, L.C. Multi-player residual advantage learning with general function approximation. Technical Report WL-TR-1065, Wright-Patterson Air Force Base, 1996.
- Harmon, M.E., Baird, L.C., and Klopff, A.H. Advantage updating applied to a differential game. In G. Tesauro, D.S. Touretzky and Leen, T.K. (eds.), *NIPS*, 1995.
- LeCun, Y., Bengio, Y., and Hinton, G. Deep learning. *Nature*, 521(7553):436–444, 2015.
- Levine, S., Finn, C., Darrell, T., and Abbeel, P. End-to-end training of deep visuomotor policies. *arXiv preprint arXiv:1504.00702*, 2015.
- Lin, L.J. *Reinforcement learning for robots using neural networks*. PhD thesis, School of Computer Science, Carnegie Mellon University, 1993.
- Maddison, C. J., Huang, A., Sutskever, I., and Silver, D. Move Evaluation in Go Using Deep Convolutional Neural Networks. In *ICLR*, 2015.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., Maria, A. De, Panneershelvam, V., Suleyman, M., Beattie, C., Petersen, S., Legg, S., Mnih, V., Kavukcuoglu, K., and Silver, D. Massively parallel methods for deep reinforcement learning. In *Deep Learning Workshop, ICML*, 2015.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. In *ICLR*, 2016.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 01 2016.
- Simonyan, K., Vedaldi, A., and Zisserman, A. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.
- Stadie, B. C., Levine, S., and Abbeel, P. Incentivizing exploration in reinforcement learning with deep predictive models. *arXiv preprint arXiv:1507.00814*, 2015.
- Sutton, R. S. and Barto, A. G. *Introduction to reinforcement learning*. MIT Press, 1998.
- Sutton, R. S., Mcallester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In *NIPS*, pp. 1057–1063, 2000.
- van Hasselt, H. Double Q-learning. *NIPS*, 23:2613–2621, 2010.
- van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double Q-learning. *arXiv preprint*

具有独立优势流的架构对于此类影响具有鲁棒性。

6. 结论

我们引入了一种新的神经网络架构，它可以解耦深度 Q 网络中的价值和优势，同时共享通用的特征学习模块。新的对决架构与一些算法改进相结合，在具有挑战性的 Atari 领域中比现有的深度 RL 方法有了显著的改进。本文提出的结果是这个流行领域的最新技术。

参考

- Ba, J., Mnih, V. 和 Kavukcuoglu, K. 具有视觉注意力的多物体识别。在 *ICLR*, 2015 年。
- 贝尔德, L.C. 优势更新。技术报告 WL-TR-93-1146, 赖特帕特森空军基地, 1993 年。
- Bellemare, M. G., Naddaf, Y., Veness, J. 和 Bowling, M. 街机学习环境: 通用代理的评估平台。 *Journal of Artificial Intelligence Research*, 47: 253–279, 2013。
- Bellemare, M. G., Ostrovski, G., Guez, A., Thomas, P. S. 和 Munos, R. 增加行动差距: 强化学习的新算子。在 2016 年的 AAAI 中出现。
- Bengio, Y., Boulanger-Lewandowski, N. 和 Pascanu, R. 优化循环网络的进展。 *ICASSP*, 第 8624–8628 页, 2013 年。
- Fukushima, K. Neocognitron: 一种自组织神经网络模型, 用于不受位置变化影响的模式识别机制。 *Biological Cybernetics*, 36: 193–202, 1980。
- Guo, X., Singh, S., Lee, H., Lewis, R. L. 和 Wang, X. 使用离线蒙特卡罗树搜索规划进行实时 Atari 游戏的深度学习。 *NIPS*, 第 3338–3346 页。2014 年。
- Harmon, M.E. 和 Baird, L.C. 具有一般函数逼近的多人剩余优势学习。技术报告 WL-TR-1065, 赖特帕特森空军基地, 1996 年。
- Harmon, M.E., Baird, L.C. 和 Klopff, A.H. 优势更新应用于微分博弈。 G. Tesauro, D.S. Touretzky 和 Leen, T. K. (编辑), *NIPS*, 1995。
- LeCun, Y., Bengio, Y. 和 Hinton, G. 深度学习。 *Nature*, 521(7553):436–444, 2015。Levine, S., Finn, C., Darrell, T. 和 Abbeel, P. 深度视觉运动策略的端到端训练。 *arXiv preprint arXiv:1504.00702*, 2015 年。
- 林, L.J. *Reinforcement learning for robots using neural networks*. 博士论文, 卡内基梅隆大学计算机科学学院, 1993 年。Maddison, C. J., Huang, A., Sutskever, I. 和 Silver, D. 使用深度卷积神经网络在围棋中进行移动评估。在 *ICLR*, 2015 年。
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. 和 Hassabis, D. 通过深度强化学习进行人类级别的控制。 *Nature*, 518 (7540): 529–533, 2015。
- Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., Maria, A. De, Panneershelvam, V., Suleyman, M., Beattie, C., Petersen, S., Legg, S., Mnih, V., Kavukcuoglu, K. 和 Silver, D. 深度强化学习的大规模并行方法。在 *Deep Learning Workshop, ICML*, 2015 年。
- Schaul, T., Quan, J., Antonoglou, I. 和 Silver, D. 优先体验重放。在 *ICLR*, 2016 年。
- Schulman, J., Moritz, P., Levine, S., Jordan, M. I. 和 Abbeel, P. 使用广义优势估计的高维连续控制。 *arXiv 预印本 arXiv:1506.02438*, 2015。
- Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T. 和 Hassabis, D. 通过深度神经网络和树搜索掌握围棋游戏。 *Nature*, 529 (7587): 484–489, 2016 年 1 月。
- Simonyan, K., Vedaldi, A. 和 Zisserman, A. 深层卷积网络: 可视化图像分类模型和显著图。 *arXiv preprint arXiv:1312.6034*, 2013 年。
- Stadie, B. C., Levine, S. 和 Abbeel, P. 利用深度预测模型激励强化学习探索。 *arXiv preprint arXiv:1507.00814*, 2015。Sutton, R.S. 和 Barto, A.G. *Introduction to reinforcement learning*. 麻省理工学院出版社, 1998 年。Sutton, R. S., McAllester, D., Singh, S. 和 Mansour, Y. 函数逼近强化学习的策略梯度方法。在 *NIPS*, 第 1057–1063 页, 2000 年。van Hasselt, H. 双 Q 学习。 *NIPS*, 23:2613–2621, 2010。van Hasselt, H., Guez, A. 和 Silver, D. 使用双 Q 学习的深度强化学习。 *arXiv preprint*

arXiv:1509.06461, 2015.

van Seijen, H., van Hasselt, H., Whiteson, S., and Wiering, M. A theoretical and empirical analysis of Expected Sarsa. In *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning*, pp. 177–184. 2009.

Watter, M., Springenberg, J. T., Boedecker, J., and Riedmiller, M. A. Embed to control: A locally linear latent dynamics model for control from raw images. In *NIPS*, 2015.

arXiv:1509.06461, 2015。

van Seijen, H.、 van Hasselt, H.、 Whiteson, S. 和 Wiering, M. 对预期 Sarsa 的理论和实证分析。

IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning, 第 177-184 页。
。 2009年。

Watter, M.、 Springenberg, J. T.、 Boedecker, J. 和 Riedmiller, M. A. 嵌入控制：用于从原始图像进行控制的局部线性潜在动力学模型。在 *NIPS*, 2015 年。

A. Double DQN Algorithm

Algorithm 1: Double DQN Algorithm.

input : $\mathcal{D}$ – empty replay buffer; θ – initial network parameters, θ^- – copy of θ
input : N_r – replay buffer maximum size; N_b – training batch size; N^- – target network replacement freq.
for *episode* $e \in \{1, 2, \dots, M\}$ **do**
 Initialize frame sequence $\mathbf{x} \leftarrow ()$
 for $t \in \{0, 1, \dots\}$ **do**
 Set state $s \leftarrow \mathbf{x}$, sample action $a \sim \pi_{\mathcal{B}}$
 Sample next frame x^t from environment $\mathcal{E}$ given (s, a) and receive reward r , and append x^t to $\mathbf{x}$
 if $|\mathbf{x}| > N_r$ **then** delete oldest frame $x_{t_{min}}$ from $\mathbf{x}$ **end**
 Set $s' \leftarrow \mathbf{x}$, and add transition tuple (s, a, r, s') to $\mathcal{D}$,
 replacing the oldest tuple **if** $|\mathcal{D}| \geq N_r$
 Sample a minibatch of N_b tuples $(s, a, r, s') \sim \text{Unif}(\mathcal{D})$
 Construct target values, one for each of the N_b tuples:
 Define $a^{\max}(s'; \theta) = \arg \max_{a'} Q(s', a'; \theta)$

$$y_j = \begin{cases} r & \text{if } s' \text{ is terminal} \\ r + \gamma Q(s', a^{\max}(s'; \theta); \theta^-), & \text{otherwise.} \end{cases}$$

 Do a gradient descent step with loss $\|y_j - Q(s, a; \theta)\|^2$
 Replace target parameters $\theta^- \leftarrow \theta$ every N^- steps
 end
end

A. 双 DQN 算法

Algorithm 1: Double DQN Algorithm.

input : $\mathcal{D}$ – empty replay buffer; θ – initial network parameters, θ^- – copy of θ
input : N_r – replay buffer maximum size; N_b – training batch size; N^- – target network replacement freq.
for *episode* $e \in \{1, 2, \dots, M\}$ **do**
 Initialize frame sequence $\mathbf{x} \leftarrow ()$
 for $t \in \{0, 1, \dots\}$ **do**
 Set state $s \leftarrow \mathbf{x}$, sample action $a \sim \pi_{\mathcal{B}}$
 Sample next frame x^t from environment $\mathcal{E}$ given (s, a) and receive reward r , and append x^t to $\mathbf{x}$
 if $|\mathbf{x}| > N_r$ **then** delete oldest frame $x_{t_{min}}$ from $\mathbf{x}$ **end**
 Set $s' \leftarrow \mathbf{x}$, and add transition tuple (s, a, r, s') to $\mathcal{D}$,
 replacing the oldest tuple if $|\mathcal{D}| \geq N_r$
 Sample a minibatch of N_b tuples $(s, a, r, s') \sim \text{Unif}(\mathcal{D})$
 Construct target values, one for each of the N_b tuples:
 Define $a^{\max}(s'; \theta) = \arg \max_{a'} Q(s', a'; \theta)$

$$y_j = \begin{cases} r & \text{if } s' \text{ is terminal} \\ r + \gamma Q(s', a^{\max}(s'; \theta); \theta^-), & \text{otherwise.} \end{cases}$$

 Do a gradient descent step with loss $\|y_j - Q(s, a; \theta)\|^2$
 Replace target parameters $\theta^- \leftarrow \theta$ every N^- steps
 end
end

Dueling Network Architectures for Deep Reinforcement Learning

Table 2. Raw scores across all games. Starting with **30 no-op** actions.

GAMES	NO. ACTIONS	RANDOM	HUMAN	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	18	227.8	7,127.7	1,620.0	3,747.7	4,461.4	4,203.8	3,941.0
Amidar	10	5.8	1,719.5	978.0	1,793.3	2,354.5	1,838.9	2,296.8
Assault	7	222.4	742.0	4,280.4	5,393.2	4,621.0	7,672.1	11,477.0
Asterix	9	210.0	8,503.3	4,359.0	17,356.5	28,188.0	31,527.0	375,080.0
Asteroids	14	719.1	47,388.7	1,364.5	734.7	2,837.7	2,654.3	1,192.7
Atlantis	4	12,850.0	29,028.1	279,987.0	106,056.0	382,572.0	357,324.0	395,762.0
Bank Heist	18	14.2	753.1	455.0	1,030.6	1,611.9	1,054.6	1,503.1
Battle Zone	18	2,360.0	37,187.5	29,900.0	31,700.0	37,150.0	31,530.0	35,520.0
Beam Rider	9	363.9	16,926.5	8,627.5	13,772.8	12,164.0	23,384.2	30,276.5
Berzerk	18	123.7	2,630.4	585.6	1,225.4	1,472.6	1,305.6	3,409.0
Bowling	6	23.1	160.7	50.4	68.1	65.5	47.9	46.7
Boxing	18	0.1	12.1	88.0	91.6	99.4	95.6	98.9
Breakout	4	1.7	30.5	385.5	418.5	345.3	373.9	366.0
Centipede	18	2,090.9	12,017.0	4,657.7	5,409.4	7,561.4	4,463.2	7,687.5
Chopper Command	18	811.0	7,387.8	6,126.0	5,809.0	11,215.0	8,600.0	13,185.0
Crazy Climber	9	10,780.5	35,829.4	110,763.0	117,282.0	143,570.0	141,161.0	162,224.0
Defender	18	2,874.5	18,688.9	23,633.0	35,338.5	42,214.0	31,286.5	41,324.5
Demon Attack	6	152.1	1,971.0	12,149.4	58,044.2	60,813.3	71,846.4	72,878.6
Double Dunk	18	-18.6	-16.4	-6.6	-5.5	0.1	18.5	-12.5
Enduro	9	0.0	860.5	729.0	1,211.8	2,258.2	2,093.0	2,306.4
Fishing Derby	18	-91.7	-38.7	-4.9	15.5	46.4	39.5	41.3
Freeway	3	0.0	29.6	30.8	33.3	0.0	33.7	33.0
Frostbite	18	65.2	4,334.7	797.4	1,683.3	4,672.8	4,380.1	7,413.0
Gopher	8	257.6	2,412.5	8,777.4	14,840.8	15,718.4	32,487.2	104,368.2
Gravitar	18	173.0	3,351.4	473.0	412.0	588.0	548.5	238.0
H.E.R.O.	18	1,027.0	30,826.4	20,437.8	20,130.2	20,818.2	23,037.7	21,036.5
Ice Hockey	18	-11.2	0.9	-1.9	-2.7	0.5	1.3	-0.4
James Bond	18	29.0	302.8	768.5	1,358.0	1,312.5	5,148.0	812.0
Kangaroo	18	52.0	3,035.0	7,259.0	12,992.0	14,854.0	16,200.0	1,792.0
Krull	18	1,598.0	2,665.5	8,422.3	7,920.5	11,451.9	9,728.0	10,374.4
Kung-Fu Master	14	258.5	22,736.3	26,059.0	29,710.0	34,294.0	39,581.0	48,375.0
Montezuma's Revenge	18	0.0	4,753.3	0.0	0.0	0.0	0.0	0.0
Ms. Pac-Man	9	307.3	6,951.6	3,085.6	2,711.4	6,283.5	6,518.7	3,327.3
Name This Game	6	2,292.3	8,049.0	8,207.8	10,616.0	11,971.1	12,270.5	15,572.5
Phoenix	8	761.4	7,242.6	8,485.2	12,252.5	23,092.2	18,992.7	70,324.3
Pitfall!	18	-229.4	6,463.7	-286.1	-29.9	0.0	-356.5	0.0
Pong	3	-20.7	14.6	19.5	20.9	21.0	20.6	20.9
Private Eye	18	24.9	69,571.3	146.7	129.7	103.0	200.0	206.0
Q*Bert	6	163.9	13,455.0	13,117.3	15,088.5	19,220.3	16,256.5	18,760.3
River Raid	18	1,338.5	17,118.0	7,377.6	14,884.5	21,162.6	14,522.3	20,607.6
Road Runner	18	11.5	7,845.0	39,544.0	44,127.0	69,524.0	57,608.0	62,151.0
Robotank	18	2.2	11.9	63.9	65.1	65.3	62.6	27.5
Seaquest	18	68.4	42,054.7	5,860.6	16,452.7	50,254.2	26,357.8	931.6
Skiing	3	-17,098.1	-4,336.9	-13,062.3	-9,021.8	-8,857.4	-9,996.9	-19,949.9
Solaris	18	1,236.3	12,326.7	3,482.8	3,067.8	2,250.8	4,309.0	133.4
Space Invaders	6	148.0	1,668.7	1,692.3	2,525.5	6,427.3	2,865.8	15,311.5
Star Gunner	18	664.0	10,250.0	54,282.0	60,142.0	89,238.0	63,302.0	125,117.0
Surround	5	-10.0	6.5	-5.6	-2.9	4.4	8.9	1.2
Tennis	18	-23.8	-8.3	12.2	-22.8	5.1	0.0	0.0
Time Pilot	10	3,568.0	5,229.2	4,870.0	8,339.0	11,666.0	9,197.0	7,553.0
Tutankham	8	11.4	167.6	68.1	218.4	211.4	204.6	245.9
Up and Down	6	533.4	11,693.2	9,989.9	22,972.2	44,939.6	16,154.1	33,879.1
Venture	18	0.0	1,187.5	163.0	98.0	497.0	54.0	48.0
Video Pinball	9	16,256.9	17,667.9	196,760.4	309,941.9	98,209.5	282,007.3	479,197.0
Wizard Of Wor	10	563.5	4,756.5	2,704.0	7,492.0	7,855.0	4,802.0	12,352.0
Yars' Revenge	18	3,092.9	54,576.9	18,098.9	11,712.6	49,622.1	11,357.0	69,618.1
Zaxxon	18	32.5	9,173.3	5,363.0	10,163.0	12,944.0	10,469.0	13,886.0

Dueling Network Architectures for Deep Reinforcement Learning

表 2. 所有游戏的原始得分。从 30 个无操作操作开始。

GAMES	NO. ACTIONS	RANDOM	HUMAN	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	18	227.8	7,127.7	1,620.0	3,747.7	4,461.4	4,203.8	3,941.0
Amidar	10	5.8	1,719.5	978.0	1,793.3	2,354.5	1,838.9	2,296.8
Assault	7	222.4	742.0	4,280.4	5,393.2	4,621.0	7,672.1	11,477.0
Asterix	9	210.0	8,503.3	4,359.0	17,356.5	28,188.0	31,527.0	375,080.0
Asteroids	14	719.1	47,388.7	1,364.5	734.7	2,837.7	2,654.3	1,192.7
Atlantis	4	12,850.0	29,028.1	279,987.0	106,056.0	382,572.0	357,324.0	395,762.0
Bank Heist	18	14.2	753.1	455.0	1,030.6	1,611.9	1,054.6	1,503.1
Battle Zone	18	2,360.0	37,187.5	29,900.0	31,700.0	37,150.0	31,530.0	35,520.0
Beam Rider	9	363.9	16,926.5	8,627.5	13,772.8	12,164.0	23,384.2	30,276.5
Berzerk	18	123.7	2,630.4	585.6	1,225.4	1,472.6	1,305.6	3,409.0
Bowling	6	23.1	160.7	50.4	68.1	65.5	47.9	46.7
Boxing	18	0.1	12.1	88.0	91.6	99.4	95.6	98.9
Breakout	4	1.7	30.5	385.5	418.5	345.3	373.9	366.0
Centipede	18	2,090.9	12,017.0	4,657.7	5,409.4	7,561.4	4,463.2	7,687.5
Chopper Command	18	811.0	7,387.8	6,126.0	5,809.0	11,215.0	8,600.0	13,185.0
Crazy Climber	9	10,780.5	35,829.4	110,763.0	117,282.0	143,570.0	141,161.0	162,224.0
Defender	18	2,874.5	18,688.9	23,633.0	35,338.5	42,214.0	31,286.5	41,324.5
Demon Attack	6	152.1	1,971.0	12,149.4	58,044.2	60,813.3	71,846.4	72,878.6
Double Dunk	18	-18.6	-16.4	-6.6	-5.5	0.1	18.5	-12.5
Enduro	9	0.0	860.5	729.0	1,211.8	2,258.2	2,093.0	2,306.4
Fishing Derby	18	-91.7	-38.7	-4.9	15.5	46.4	39.5	41.3
Freeway	3	0.0	29.6	30.8	33.3	0.0	33.7	33.0
Frostbite	18	65.2	4,334.7	797.4	1,683.3	4,672.8	4,380.1	7,413.0
Gopher	8	257.6	2,412.5	8,777.4	14,840.8	15,718.4	32,487.2	104,368.2
Gravitar	18	173.0	3,351.4	473.0	412.0	588.0	548.5	238.0
H.E.R.O.	18	1,027.0	30,826.4	20,437.8	20,130.2	20,818.2	23,037.7	21,036.5
Ice Hockey	18	-11.2	0.9	-1.9	-2.7	0.5	1.3	-0.4
James Bond	18	29.0	302.8	768.5	1,358.0	1,312.5	5,148.0	812.0
Kangaroo	18	52.0	3,035.0	7,259.0	12,992.0	14,854.0	16,200.0	1,792.0
Krull	18	1,598.0	2,665.5	8,422.3	7,920.5	11,451.9	9,728.0	10,374.4
Kung-Fu Master	14	258.5	22,736.3	26,059.0	29,710.0	34,294.0	39,581.0	48,375.0
Montezuma's Revenge	18	0.0	4,753.3	0.0	0.0	0.0	0.0	0.0
Ms. Pac-Man	9	307.3	6,951.6	3,085.6	2,711.4	6,283.5	6,518.7	3,327.3
Name This Game	6	2,292.3	8,049.0	8,207.8	10,616.0	11,971.1	12,270.5	15,572.5
Phoenix	8	761.4	7,242.6	8,485.2	12,252.5	23,092.2	18,992.7	70,324.3
Pitfall!	18	-229.4	6,463.7	-286.1	-29.9	0.0	-356.5	0.0
Pong	3	-20.7	14.6	19.5	20.9	21.0	20.6	20.9
Private Eye	18	24.9	69,571.3	146.7	129.7	103.0	200.0	206.0
Q*Bert	6	163.9	13,455.0	13,117.3	15,088.5	19,220.3	16,256.5	18,760.3
River Raid	18	1,338.5	17,118.0	7,377.6	14,884.5	21,162.6	14,522.3	20,607.6
Road Runner	18	11.5	7,845.0	39,544.0	44,127.0	69,524.0	57,608.0	62,151.0
Robotank	18	2.2	11.9	63.9	65.1	65.3	62.6	27.5
Seaquest	18	68.4	42,054.7	5,860.6	16,452.7	50,254.2	26,357.8	931.6
Skiing	3	-17,098.1	-4,336.9	-13,062.3	-9,021.8	-8,857.4	-9,996.9	-19,949.9
Solaris	18	1,236.3	12,326.7	3,482.8	3,067.8	2,250.8	4,309.0	133.4
Space Invaders	6	148.0	1,668.7	1,692.3	2,525.5	6,427.3	2,865.8	15,311.5
Star Gunner	18	664.0	10,250.0	54,282.0	60,142.0	89,238.0	63,302.0	125,117.0
Surround	5	-10.0	6.5	-5.6	-2.9	4.4	8.9	1.2
Tennis	18	-23.8	-8.3	12.2	-22.8	5.1	0.0	0.0
Time Pilot	10	3,568.0	5,229.2	4,870.0	8,339.0	11,666.0	9,197.0	7,553.0
Tutankham	8	11.4	167.6	68.1	218.4	211.4	204.6	245.9
Up and Down	6	533.4	11,693.2	9,989.9	22,972.2	44,939.6	16,154.1	33,879.1
Venture	18	0.0	1,187.5	163.0	98.0	497.0	54.0	48.0
Video Pinball	9	16,256.9	17,667.9	196,760.4	309,941.9	98,209.5	282,007.3	479,197.0
Wizard Of Wor	10	563.5	4,756.5	2,704.0	7,492.0	7,855.0	4,802.0	12,352.0
Yars' Revenge	18	3,092.9	54,576.9	18,098.9	11,712.6	49,622.1	11,357.0	69,618.1
Zaxxon	18	32.5	9,173.3	5,363.0	10,163.0	12,944.0	10,469.0	13,886.0

Dueling Network Architectures for Deep Reinforcement Learning

Table 3. Raw scores across all games. Starting with **Human starts**.

GAMES	NO. ACTIONS	RANDOM	HUMAN	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	18	128.3	6,371.3	634.0	1,033.4	1,486.5	1,334.7	823.7
Amidar	10	11.8	1,540.4	178.4	169.1	172.7	129.1	238.4
Assault	7	166.9	628.9	3,489.3	6,060.8	3,994.8	6,548.9	10,950.6
Asterix	9	164.5	7,536.0	3,170.5	16,837.0	15,840.0	22,484.5	364,200.0
Asteroids	14	871.3	36,517.3	1,458.7	1,193.2	2,035.4	1,745.1	1,021.9
Atlantis	4	13,463.0	26,575.0	292,491.0	319,688.0	445,360.0	330,647.0	423,252.0
Bank Heist	18	21.7	644.5	312.7	886.0	1,129.3	876.6	1,004.6
Battle Zone	18	3,560.0	33,030.0	23,750.0	24,740.0	31,320.0	25,520.0	30,650.0
Beam Rider	9	254.6	14,961.0	9,743.2	17,417.2	14,591.3	31,181.3	37,412.2
Berzerk	18	196.1	2,237.5	493.4	1,011.1	910.6	865.9	2,178.6
Bowling	6	35.2	146.5	56.5	69.6	65.7	52.0	50.4
Boxing	18	-1.5	9.6	70.3	73.5	77.3	72.3	79.2
Breakout	4	1.6	27.9	354.5	368.9	411.6	343.0	354.6
Centipede	18	1,925.5	10,321.9	3,973.9	3,853.5	4,881.0	3,489.1	5,570.2
Chopper Command	18	644.0	8,930.0	5,017.0	3,495.0	3,784.0	4,635.0	8,058.0
Crazy Climber	9	9,337.0	32,667.0	98,128.0	113,782.0	124,566.0	127,512.0	127,853.0
Defender	18	1,965.5	14,296.0	15,917.5	27,510.0	33,996.0	23,666.5	34,415.0
Demon Attack	6	208.3	3,442.8	12,550.7	69,803.4	56,322.8	61,277.5	73,371.3
Double Dunk	18	-16.0	-14.4	-6.0	-0.3	-0.8	16.0	-10.7
Enduro	9	-81.8	740.2	626.7	1,216.6	2,077.4	1,831.0	2,223.9
Fishing Derby	18	-77.1	5.1	-1.6	3.2	-4.1	9.8	17.0
Freeway	3	0.1	25.6	26.9	28.8	0.2	28.9	28.2
Frostbite	18	66.4	4,202.8	496.1	1,448.1	2,332.4	3,510.0	4,038.4
Gopher	8	250.0	2,311.0	8,190.4	15,253.0	20,051.4	34,858.8	105,148.4
Gravitar	18	245.5	3,116.0	298.0	200.5	297.0	269.5	167.0
H.E.R.O.	18	1,580.3	25,839.4	14,992.9	14,892.5	15,207.9	20,889.9	15,459.2
Ice Hockey	18	-9.7	0.5	-1.6	-2.5	-1.3	-0.2	0.5
James Bond	18	33.5	368.5	697.5	573.0	835.5	3,961.0	585.0
Kangaroo	18	100.0	2,739.0	4,496.0	11,204.0	10,334.0	12,185.0	861.0
Krull	18	1,151.9	2,109.1	6,206.0	6,796.1	8,051.6	6,872.8	7,658.6
Kung-Fu Master	14	304.0	20,786.8	20,882.0	30,207.0	24,288.0	31,676.0	37,484.0
Montezuma's Revenge	18	25.0	4,182.0	47.0	42.0	22.0	51.0	24.0
Ms. Pac-Man	9	197.8	15,375.0	1,092.3	1,241.3	2,250.6	1,865.9	1,007.8
Name This Game	6	1,747.8	6,796.0	6,738.8	8,960.3	11,185.1	10,497.6	13,637.9
Phoenix	8	1,134.4	6,686.2	7,484.8	12,366.5	20,410.5	16,903.6	63,597.0
Pitfall!	18	-348.8	5,998.9	-113.2	-186.7	-46.9	-427.0	-243.6
Pong	3	-18.0	15.5	18.0	19.1	18.8	18.9	18.4
Private Eye	18	662.8	64,169.1	207.9	-575.5	292.6	670.7	1,277.6
Q*Bert	6	183.0	12,085.0	9,271.5	11,020.8	14,175.8	9,944.0	14,063.0
River Raid	18	588.3	14,382.2	4,748.5	10,838.4	16,569.4	11,807.2	16,496.8
Road Runner	18	200.0	6,878.0	35,215.0	43,156.0	58,549.0	52,264.0	54,630.0
Robotank	18	2.4	8.9	58.7	59.1	62.0	56.2	24.7
Seaquest	18	215.5	40,425.8	4,216.7	14,498.0	37,361.6	25,463.7	1,431.2
Skiing	3	-15,287.4	-3,686.6	-12,142.1	-11,490.4	-11,928.0	-10,169.1	-18,955.8
Solaris	18	2,047.2	11,032.6	1,295.4	810.0	1,768.4	2,272.8	280.6
Space Invaders	6	182.6	1,464.9	1,293.8	2,628.7	5,993.1	3,912.1	8,978.0
Star Gunner	18	697.0	9,528.0	52,970.0	58,365.0	90,804.0	61,582.0	127,073.0
Surround	5	-9.7	5.4	-6.0	1.9	4.0	5.9	-0.2
Tennis	18	-21.4	-6.7	11.1	-7.8	4.4	-5.3	-13.2
Time Pilot	10	3,273.0	5,650.0	4,786.0	6,608.0	6,601.0	5,963.0	4,871.0
Tutankham	8	12.7	138.3	45.6	92.2	48.0	56.9	108.6
Up and Down	6	707.2	9,896.1	8,038.5	19,086.9	24,759.2	12,157.4	22,681.3
Venture	18	18.0	1,039.0	136.0	21.0	200.0	94.0	29.0
Video Pinball	9	20,452.0	15,641.1	154,414.1	367,823.7	110,976.2	295,972.8	447,408.6
Wizard Of Wor	10	804.0	4,556.0	1,609.0	6,201.0	7,054.0	5,727.0	10,471.0
Yars' Revenge	18	1,476.9	47,135.2	4,577.5	6,270.6	25,976.5	4,687.4	58,145.9
Zaxxon	18	475.0	8,443.0	4,412.0	8,593.0	10,164.0	9,474.0	11,320.0

表 3. 所有游戏的原始得分。从人类开始。

GAMES	NO. ACTIONS	RANDOM	HUMAN	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	18	128.3	6,371.3	634.0	1,033.4	1,486.5	1,334.7	823.7
Amidar	10	11.8	1,540.4	178.4	169.1	172.7	129.1	238.4
Assault	7	166.9	628.9	3,489.3	6,060.8	3,994.8	6,548.9	10,950.6
Asterix	9	164.5	7,536.0	3,170.5	16,837.0	15,840.0	22,484.5	364,200.0
Asteroids	14	871.3	36,517.3	1,458.7	1,193.2	2,035.4	1,745.1	1,021.9
Atlantis	4	13,463.0	26,575.0	292,491.0	319,688.0	445,360.0	330,647.0	423,252.0
Bank Heist	18	21.7	644.5	312.7	886.0	1,129.3	876.6	1,004.6
Battle Zone	18	3,560.0	33,030.0	23,750.0	24,740.0	31,320.0	25,520.0	30,650.0
Beam Rider	9	254.6	14,961.0	9,743.2	17,417.2	14,591.3	31,181.3	37,412.2
Berzerk	18	196.1	2,237.5	493.4	1,011.1	910.6	865.9	2,178.6
Bowling	6	35.2	146.5	56.5	69.6	65.7	52.0	50.4
Boxing	18	-1.5	9.6	70.3	73.5	77.3	72.3	79.2
Breakout	4	1.6	27.9	354.5	368.9	411.6	343.0	354.6
Centipede	18	1,925.5	10,321.9	3,973.9	3,853.5	4,881.0	3,489.1	5,570.2
Chopper Command	18	644.0	8,930.0	5,017.0	3,495.0	3,784.0	4,635.0	8,058.0
Crazy Climber	9	9,337.0	32,667.0	98,128.0	113,782.0	124,566.0	127,512.0	127,853.0
Defender	18	1,965.5	14,296.0	15,917.5	27,510.0	33,996.0	23,666.5	34,415.0
Demon Attack	6	208.3	3,442.8	12,550.7	69,803.4	56,322.8	61,277.5	73,371.3
Double Dunk	18	-16.0	-14.4	-6.0	-0.3	-0.8	16.0	-10.7
Enduro	9	-81.8	740.2	626.7	1,216.6	2,077.4	1,831.0	2,223.9
Fishing Derby	18	-77.1	5.1	-1.6	3.2	-4.1	9.8	17.0
Freeway	3	0.1	25.6	26.9	28.8	0.2	28.9	28.2
Frostbite	18	66.4	4,202.8	496.1	1,448.1	2,332.4	3,510.0	4,038.4
Gopher	8	250.0	2,311.0	8,190.4	15,253.0	20,051.4	34,858.8	105,148.4
Gravitar	18	245.5	3,116.0	298.0	200.5	297.0	269.5	167.0
H.E.R.O.	18	1,580.3	25,839.4	14,992.9	14,892.5	15,207.9	20,889.9	15,459.2
Ice Hockey	18	-9.7	0.5	-1.6	-2.5	-1.3	-0.2	0.5
James Bond	18	33.5	368.5	697.5	573.0	835.5	3,961.0	585.0
Kangaroo	18	100.0	2,739.0	4,496.0	11,204.0	10,334.0	12,185.0	861.0
Krull	18	1,151.9	2,109.1	6,206.0	6,796.1	8,051.6	6,872.8	7,658.6
Kung-Fu Master	14	304.0	20,786.8	20,882.0	30,207.0	24,288.0	31,676.0	37,484.0
Montezuma's Revenge	18	25.0	4,182.0	47.0	42.0	22.0	51.0	24.0
Ms. Pac-Man	9	197.8	15,375.0	1,092.3	1,241.3	2,250.6	1,865.9	1,007.8
Name This Game	6	1,747.8	6,796.0	6,738.8	8,960.3	11,185.1	10,497.6	13,637.9
Phoenix	8	1,134.4	6,686.2	7,484.8	12,366.5	20,410.5	16,903.6	63,597.0
Pitfall!	18	-348.8	5,998.9	-113.2	-186.7	-46.9	-427.0	-243.6
Pong	3	-18.0	15.5	18.0	19.1	18.8	18.9	18.4
Private Eye	18	662.8	64,169.1	207.9	-575.5	292.6	670.7	1,277.6
Q*Bert	6	183.0	12,085.0	9,271.5	11,020.8	14,175.8	9,944.0	14,063.0
River Raid	18	588.3	14,382.2	4,748.5	10,838.4	16,569.4	11,807.2	16,496.8
Road Runner	18	200.0	6,878.0	35,215.0	43,156.0	58,549.0	52,264.0	54,630.0
Robotank	18	2.4	8.9	58.7	59.1	62.0	56.2	24.7
Seaquest	18	215.5	40,425.8	4,216.7	14,498.0	37,361.6	25,463.7	1,431.2
Skiing	3	-15,287.4	-3,686.6	-12,142.1	-11,490.4	-11,928.0	-10,169.1	-18,955.8
Solaris	18	2,047.2	11,032.6	1,295.4	810.0	1,768.4	2,272.8	280.6
Space Invaders	6	182.6	1,464.9	1,293.8	2,628.7	5,993.1	3,912.1	8,978.0
Star Gunner	18	697.0	9,528.0	52,970.0	58,365.0	90,804.0	61,582.0	127,073.0
Surround	5	-9.7	5.4	-6.0	1.9	4.0	5.9	-0.2
Tennis	18	-21.4	-6.7	11.1	-7.8	4.4	-5.3	-13.2
Time Pilot	10	3,273.0	5,650.0	4,786.0	6,608.0	6,601.0	5,963.0	4,871.0
Tutankham	8	12.7	138.3	45.6	92.2	48.0	56.9	108.6
Up and Down	6	707.2	9,896.1	8,038.5	19,086.9	24,759.2	12,157.4	22,681.3
Venture	18	18.0	1,039.0	136.0	21.0	200.0	94.0	29.0
Video Pinball	9	20,452.0	15,641.1	154,414.1	367,823.7	110,976.2	295,972.8	447,408.6
Wizard Of Wor	10	804.0	4,556.0	1,609.0	6,201.0	7,054.0	5,727.0	10,471.0
Yars' Revenge	18	1,476.9	47,135.2	4,577.5	6,270.6	25,976.5	4,687.4	58,145.9
Zaxxon	18	475.0	8,443.0	4,412.0	8,593.0	10,164.0	9,474.0	11,320.0

Table 4. Normalized scores across all games. Starting with **30 no-op** actions.

GAMES	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	20.2%	51.0%	61.4%	57.6%	53.8%
Amidar	56.7%	104.3%	137.1%	107.0%	133.7%
Assault	781.0%	995.1%	846.5%	1433.7%	2166.0%
Asterix	50.0%	206.8%	337.4%	377.6%	4520.1%
Asteroids	1.4%	0.0%	4.5%	4.1%	1.0%
Atlantis	1651.2%	576.1%	2285.3%	2129.3%	2366.9%
Bank Heist	59.7%	137.6%	216.2%	140.8%	201.5%
Battle Zone	79.1%	84.2%	99.9%	83.8%	95.2%
Beam Rider	49.9%	81.0%	71.2%	139.0%	180.6%
Berzerk	18.4%	44.0%	53.8%	47.2%	131.1%
Bowling	19.8%	32.7%	30.8%	18.0%	17.1%
Boxing	732.5%	762.1%	827.1%	795.5%	823.1%
Breakout	1334.5%	1449.2%	1194.5%	1294.3%	1266.6%
Centipede	25.9%	33.4%	55.1%	23.9%	56.4%
Chopper Command	80.8%	76.0%	158.2%	118.4%	188.1%
Crazy Climber	399.1%	425.2%	530.1%	520.5%	604.6%
Defender	131.3%	205.3%	248.8%	179.7%	243.1%
Demon Attack	659.6%	3182.8%	3335.0%	3941.6%	3998.3%
Double Dunk	557.7%	607.9%	866.5%	1723.3%	280.5%
Enduro	84.7%	140.8%	262.4%	243.2%	268.0%
Fishing Derby	163.8%	202.4%	260.7%	247.7%	251.1%
Freeway	104.0%	112.5%	0.1%	114.0%	111.3%
Frostbite	17.1%	37.9%	107.9%	101.1%	172.1%
Gopher	395.4%	676.7%	717.5%	1495.6%	4831.3%
Gravitar	9.4%	7.5%	13.1%	11.8%	2.0%
H.E.R.O.	65.1%	64.1%	66.4%	73.9%	67.1%
Ice Hockey	76.9%	70.0%	96.4%	103.2%	89.6%
James Bond	270.1%	485.4%	468.8%	1869.8%	286.0%
Kangaroo	241.6%	433.8%	496.2%	541.3%	58.3%
Krull	639.3%	592.3%	923.1%	761.6%	822.2%
Kung-Fu Master	114.8%	131.0%	151.4%	174.9%	214.1%
Montezuma's Revenge	0.0%	0.0%	0.0%	0.0%	0.0%
Ms. Pac-Man	41.8%	36.2%	89.9%	93.5%	45.5%
Name This Game	102.8%	144.6%	168.1%	173.3%	230.7%
Phoenix	119.2%	177.3%	344.5%	281.3%	1073.3%
Pitfall!	-0.8%	3.0%	3.4%	-1.9%	3.4%
Pong	114.0%	117.8%	118.2%	117.1%	118.0%
Private Eye	0.2%	0.2%	0.1%	0.3%	0.3%
Q*Bert	97.5%	112.3%	143.4%	121.1%	139.9%
River Raid	38.3%	85.8%	125.6%	83.6%	122.1%
Road Runner	504.7%	563.2%	887.4%	735.3%	793.3%
Robotank	631.5%	643.7%	645.1%	617.5%	259.5%
Seaquest	13.8%	39.0%	119.5%	62.6%	2.1%
Skiing	31.6%	63.3%	64.6%	55.6%	-22.3%
Solaris	20.3%	16.5%	9.1%	27.7%	-9.9%
Space Invaders	101.6%	156.3%	412.9%	178.7%	997.2%
Star Gunner	559.3%	620.5%	924.0%	653.4%	1298.3%
Surround	26.5%	43.2%	86.9%	114.6%	67.6%
Tennis	231.3%	6.8%	186.2%	153.2%	153.2%
Time Pilot	78.4%	287.2%	487.5%	338.9%	239.9%
Tutankham	36.3%	132.5%	128.1%	123.7%	150.1%
Up and Down	84.7%	201.1%	397.9%	140.0%	298.8%
Venture	13.7%	8.3%	41.9%	4.5%	4.0%
Video Pinball	1113.7%	1754.3%	555.9%	1596.2%	2712.2%
Wizard Of Wor	51.0%	165.2%	173.9%	101.1%	281.1%
Yars' Revenge	29.1%	16.7%	90.4%	16.1%	129.2%
Zaxxon	58.3%	110.8%	141.3%	114.2%	151.6%

Table 4. Normalized scores across all games. Starting with **30 no-op** actions.

GAMES	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	20.2%	51.0%	61.4%	57.6%	53.8%
Amidar	56.7%	104.3%	137.1%	107.0%	133.7%
Assault	781.0%	995.1%	846.5%	1433.7%	2166.0%
Asterix	50.0%	206.8%	337.4%	377.6%	4520.1%
Asteroids	1.4%	0.0%	4.5%	4.1%	1.0%
Atlantis	1651.2%	576.1%	2285.3%	2129.3%	2366.9%
Bank Heist	59.7%	137.6%	216.2%	140.8%	201.5%
Battle Zone	79.1%	84.2%	99.9%	83.8%	95.2%
Beam Rider	49.9%	81.0%	71.2%	139.0%	180.6%
Berzerk	18.4%	44.0%	53.8%	47.2%	131.1%
Bowling	19.8%	32.7%	30.8%	18.0%	17.1%
Boxing	732.5%	762.1%	827.1%	795.5%	823.1%
Breakout	1334.5%	1449.2%	1194.5%	1294.3%	1266.6%
Centipede	25.9%	33.4%	55.1%	23.9%	56.4%
Chopper Command	80.8%	76.0%	158.2%	118.4%	188.1%
Crazy Climber	399.1%	425.2%	530.1%	520.5%	604.6%
Defender	131.3%	205.3%	248.8%	179.7%	243.1%
Demon Attack	659.6%	3182.8%	3335.0%	3941.6%	3998.3%
Double Dunk	557.7%	607.9%	866.5%	1723.3%	280.5%
Enduro	84.7%	140.8%	262.4%	243.2%	268.0%
Fishing Derby	163.8%	202.4%	260.7%	247.7%	251.1%
Freeway	104.0%	112.5%	0.1%	114.0%	111.3%
Frostbite	17.1%	37.9%	107.9%	101.1%	172.1%
Gopher	395.4%	676.7%	717.5%	1495.6%	4831.3%
Gravitar	9.4%	7.5%	13.1%	11.8%	2.0%
H.E.R.O.	65.1%	64.1%	66.4%	73.9%	67.1%
Ice Hockey	76.9%	70.0%	96.4%	103.2%	89.6%
James Bond	270.1%	485.4%	468.8%	1869.8%	286.0%
Kangaroo	241.6%	433.8%	496.2%	541.3%	58.3%
Krull	639.3%	592.3%	923.1%	761.6%	822.2%
Kung-Fu Master	114.8%	131.0%	151.4%	174.9%	214.1%
Montezuma's Revenge	0.0%	0.0%	0.0%	0.0%	0.0%
Ms. Pac-Man	41.8%	36.2%	89.9%	93.5%	45.5%
Name This Game	102.8%	144.6%	168.1%	173.3%	230.7%
Phoenix	119.2%	177.3%	344.5%	281.3%	1073.3%
Pitfall!	-0.8%	3.0%	3.4%	-1.9%	3.4%
Pong	114.0%	117.8%	118.2%	117.1%	118.0%
Private Eye	0.2%	0.2%	0.1%	0.3%	0.3%
Q*Bert	97.5%	112.3%	143.4%	121.1%	139.9%
River Raid	38.3%	85.8%	125.6%	83.6%	122.1%
Road Runner	504.7%	563.2%	887.4%	735.3%	793.3%
Robotank	631.5%	643.7%	645.1%	617.5%	259.5%
Seaquest	13.8%	39.0%	119.5%	62.6%	2.1%
Skiing	31.6%	63.3%	64.6%	55.6%	-22.3%
Solaris	20.3%	16.5%	9.1%	27.7%	-9.9%
Space Invaders	101.6%	156.3%	412.9%	178.7%	997.2%
Star Gunner	559.3%	620.5%	924.0%	653.4%	1298.3%
Surround	26.5%	43.2%	86.9%	114.6%	67.6%
Tennis	231.3%	6.8%	186.2%	153.2%	153.2%
Time Pilot	78.4%	287.2%	487.5%	338.9%	239.9%
Tutankham	36.3%	132.5%	128.1%	123.7%	150.1%
Up and Down	84.7%	201.1%	397.9%	140.0%	298.8%
Venture	13.7%	8.3%	41.9%	4.5%	4.0%
Video Pinball	1113.7%	1754.3%	555.9%	1596.2%	2712.2%
Wizard Of Wor	51.0%	165.2%	173.9%	101.1%	281.1%
Yars' Revenge	29.1%	16.7%	90.4%	16.1%	129.2%
Zaxxon	58.3%	110.8%	141.3%	114.2%	151.6%

Table 5. Normalized scores across all games. Starting with **Human Starts**.

GAMES	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	8.1%	14.5%	21.8%	19.3%	11.1%
Amidar	10.9%	10.3%	10.5%	7.7%	14.8%
Assault	719.2%	1275.9%	828.6%	1381.5%	2334.4%
Asterix	40.8%	226.2%	212.7%	302.8%	4938.4%
Asteroids	1.6%	0.9%	3.3%	2.5%	0.4%
Atlantis	2128.0%	2335.5%	3293.9%	2419.0%	3125.3%
Bank Heist	46.7%	138.8%	177.8%	137.3%	157.8%
Battle Zone	68.5%	71.9%	94.2%	74.5%	91.9%
Beam Rider	64.5%	116.7%	97.5%	210.3%	252.7%
Berzerk	14.6%	39.9%	35.0%	32.8%	97.1%
Bowling	19.2%	30.9%	27.5%	15.1%	13.7%
Boxing	648.2%	677.0%	711.2%	666.7%	728.5%
Breakout	1341.9%	1396.7%	1559.0%	1298.3%	1342.4%
Centipede	24.4%	23.0%	35.2%	18.6%	43.4%
Chopper Command	52.8%	34.4%	37.9%	48.2%	89.5%
Crazy Climber	380.6%	447.7%	493.9%	506.5%	508.0%
Defender	113.2%	207.2%	259.8%	176.0%	263.2%
Demon Attack	381.6%	2151.6%	1734.8%	1888.0%	2261.9%
Double Dunk	622.5%	982.5%	948.7%	1998.7%	328.7%
Enduro	86.2%	158.0%	262.7%	232.7%	280.5%
Fishing Derby	91.8%	97.7%	88.8%	105.7%	114.5%
Freeway	105.1%	112.5%	0.6%	113.1%	110.2%
Frostbite	10.4%	33.4%	54.8%	83.2%	96.0%
Gopher	385.3%	727.9%	960.8%	1679.2%	5089.7%
Gravitar	1.8%	-1.6%	1.8%	0.8%	-2.7%
H.E.R.O.	55.3%	54.9%	56.2%	79.6%	57.2%
Ice Hockey	79.2%	70.8%	82.4%	92.5%	99.4%
James Bond	198.2%	161.0%	239.4%	1172.4%	164.6%
Kangaroo	166.6%	420.8%	387.8%	457.9%	28.8%
Krull	528.0%	589.7%	720.8%	597.7%	679.8%
Kung-Fu Master	100.5%	146.0%	117.1%	153.2%	181.5%
Montezuma's Revenge	0.5%	0.4%	-0.1%	0.6%	-0.0%
Ms. Pac-Man	5.9%	6.9%	13.5%	11.0%	5.3%
Name This Game	98.9%	142.9%	186.9%	173.3%	235.5%
Phoenix	114.4%	202.3%	347.2%	284.0%	1125.1%
Pitfall!	3.7%	2.6%	4.8%	-1.2%	1.7%
Pong	107.6%	110.9%	110.0%	110.1%	108.6%
Private Eye	-0.7%	-1.9%	-0.6%	0.0%	1.0%
Q*Bert	76.4%	91.1%	117.6%	82.0%	116.6%
River Raid	30.2%	74.3%	115.9%	81.3%	115.3%
Road Runner	524.3%	643.2%	873.7%	779.6%	815.1%
Robotank	863.3%	868.7%	913.3%	824.4%	341.1%
Seaquest	10.0%	35.5%	92.4%	62.8%	3.0%
Skiing	27.1%	32.7%	29.0%	44.1%	-31.6%
Solaris	-8.4%	-13.8%	-3.1%	2.5%	-19.7%
Space Invaders	86.7%	190.8%	453.1%	290.8%	685.9%
Star Gunner	591.9%	653.0%	1020.3%	689.4%	1431.0%
Surround	24.7%	76.9%	91.1%	103.2%	62.9%
Tennis	220.8%	92.1%	175.0%	109.6%	55.6%
Time Pilot	63.7%	140.3%	140.0%	113.2%	67.2%
Tutankham	26.2%	63.3%	28.1%	35.2%	76.4%
Up and Down	79.8%	200.0%	261.8%	124.6%	239.1%
Venture	11.6%	0.3%	17.8%	7.4%	1.1%
Video Pinball	987.2%	2351.6%	709.5%	1892.3%	2860.5%
Wizard Of Wor	21.5%	143.8%	166.6%	131.2%	257.6%
Yars' Revenge	6.8%	10.5%	53.7%	7.0%	124.1%
Zaxxon	49.4%	101.9%	121.6%	112.9%	136.1%
Mean	219.6%	332.9%	343.8%	386.7%	567.0%
Median	68.5%	110.9%	117.1%	112.9%	115.3%

表 5. 所有游戏的标准化得分。从人类开始。

GAMES	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.
Alien	8.1%	14.5%	21.8%	19.3%	11.1%
Amidar	10.9%	10.3%	10.5%	7.7%	14.8%
Assault	719.2%	1275.9%	828.6%	1381.5%	2334.4%
Asterix	40.8%	226.2%	212.7%	302.8%	4938.4%
Asteroids	1.6%	0.9%	3.3%	2.5%	0.4%
Atlantis	2128.0%	2335.5%	3293.9%	2419.0%	3125.3%
Bank Heist	46.7%	138.8%	177.8%	137.3%	157.8%
Battle Zone	68.5%	71.9%	94.2%	74.5%	91.9%
Beam Rider	64.5%	116.7%	97.5%	210.3%	252.7%
Berzerk	14.6%	39.9%	35.0%	32.8%	97.1%
Bowling	19.2%	30.9%	27.5%	15.1%	13.7%
Boxing	648.2%	677.0%	711.2%	666.7%	728.5%
Breakout	1341.9%	1396.7%	1559.0%	1298.3%	1342.4%
Centipede	24.4%	23.0%	35.2%	18.6%	43.4%
Chopper Command	52.8%	34.4%	37.9%	48.2%	89.5%
Crazy Climber	380.6%	447.7%	493.9%	506.5%	508.0%
Defender	113.2%	207.2%	259.8%	176.0%	263.2%
Demon Attack	381.6%	2151.6%	1734.8%	1888.0%	2261.9%
Double Dunk	622.5%	982.5%	948.7%	1998.7%	328.7%
Enduro	86.2%	158.0%	262.7%	232.7%	280.5%
Fishing Derby	91.8%	97.7%	88.8%	105.7%	114.5%
Freeway	105.1%	112.5%	0.6%	113.1%	110.2%
Frostbite	10.4%	33.4%	54.8%	83.2%	96.0%
Gopher	385.3%	727.9%	960.8%	1679.2%	5089.7%
Gravitar	1.8%	-1.6%	1.8%	0.8%	-2.7%
H.E.R.O.	55.3%	54.9%	56.2%	79.6%	57.2%
Ice Hockey	79.2%	70.8%	82.4%	92.5%	99.4%
James Bond	198.2%	161.0%	239.4%	1172.4%	164.6%
Kangaroo	166.6%	420.8%	387.8%	457.9%	28.8%
Krull	528.0%	589.7%	720.8%	597.7%	679.8%
Kung-Fu Master	100.5%	146.0%	117.1%	153.2%	181.5%
Montezuma's Revenge	0.5%	0.4%	-0.1%	0.6%	-0.0%
Ms. Pac-Man	5.9%	6.9%	13.5%	11.0%	5.3%
Name This Game	98.9%	142.9%	186.9%	173.3%	235.5%
Phoenix	114.4%	202.3%	347.2%	284.0%	1125.1%
Pitfall!	3.7%	2.6%	4.8%	-1.2%	1.7%
Pong	107.6%	110.9%	110.0%	110.1%	108.6%
Private Eye	-0.7%	-1.9%	-0.6%	0.0%	1.0%
Q*Bert	76.4%	91.1%	117.6%	82.0%	116.6%
River Raid	30.2%	74.3%	115.9%	81.3%	115.3%
Road Runner	524.3%	643.2%	873.7%	779.6%	815.1%
Robotank	863.3%	868.7%	913.3%	824.4%	341.1%
Seaquest	10.0%	35.5%	92.4%	62.8%	3.0%
Skiing	27.1%	32.7%	29.0%	44.1%	-31.6%
Solaris	-8.4%	-13.8%	-3.1%	2.5%	-19.7%
Space Invaders	86.7%	190.8%	453.1%	290.8%	685.9%
Star Gunner	591.9%	653.0%	1020.3%	689.4%	1431.0%
Surround	24.7%	76.9%	91.1%	103.2%	62.9%
Tennis	220.8%	92.1%	175.0%	109.6%	55.6%
Time Pilot	63.7%	140.3%	140.0%	113.2%	67.2%
Tutankham	26.2%	63.3%	28.1%	35.2%	76.4%
Up and Down	79.8%	200.0%	261.8%	124.6%	239.1%
Venture	11.6%	0.3%	17.8%	7.4%	1.1%
Video Pinball	987.2%	2351.6%	709.5%	1892.3%	2860.5%
Wizard Of Wor	21.5%	143.8%	166.6%	131.2%	257.6%
Yars' Revenge	6.8%	10.5%	53.7%	7.0%	124.1%
Zaxxon	49.4%	101.9%	121.6%	112.9%	136.1%
Mean	219.6%	332.9%	343.8%	386.7%	567.0%
Median	68.5%	110.9%	117.1%	112.9%	115.3%